UNIVERSITY OF MINES AND TECHNOLOGY, TARKWA

FACULTY OF COMPUTING AND MATHEMATICAL SCIENCES

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

A PROJECT REPORT ENTITLED

DYNAMIC CONTEXT-AWARE TRIES FOR KNOWLEDGE GRAPH-CONSTRAINED LARGE LANGUAGE MODEL GENERATION

BY

BERNARD KIRK ADJANOR KATAMANSO

ERICA AMONOR

JOSEPH OSEI NYARKO

JESSICA AFUA ETORNAM NSAFOAH

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE OF BACHELOR OF SCIENCE IN COMPUTER SCIENCE AND ENGINEERING

THESIS SUPERVISOR

.....

DR ERIC AFFUM

TARKWA, GHANA

APRIL 2026

DECLARATION

We, BERNARD KIRK ADJANOR KATAMANSO, ERICA AMONOR, JOSEPH OSEI NYARKO and JESSICA AFUA ETORNAM NSAFOAH, declare that this project work is our own original work. It is being submitted for the degree of Bachelor of Science in Computer Science and Engineering at the University of Mines and Technology (UMaT), Tarkwa. It has not been submitted, either in whole or in part, for any degree or examination at any other university.

Signature of Student: _____

BERNARD KIRK ADJANOR KATAMANSO (FOE.41.008.088.22)

Signature of Student: _____

AMONOR ERICA (FOE.41.008.030.22)

Signature of Student: _____

JOSEPH OSEI NYARKO (FOE.41.008.113.22)

Signature of Student: _____

JESSICA AFUA ETORNAM NSAFOAH (FOE.41.008.107.22)

_____ day of _____ (year) _____

ABSTRACT

Though large language models handle language well, they sometimes state incorrect information with high confidence. These false outputs, known as hallucinations, are particularly problematic when answering questions using knowledge graphs, since accuracy requires tracing true connections across structured data. To reduce errors, current techniques such as retrieval-augmented generation link model output to established facts in external sources. Methods like Graph-Constrained Reasoning (GCR) and Decoding on Graphs (DoG) go further by restricting which answers are possible based on predefined graph paths. Yet these systems lock in their constraints before generating a response. Even as the meaning of what is being said evolves during reasoning, the set of allowed paths stays unchanged. Because of this, routes through the graph that are unrelated to the question remain available throughout generation, and the model must sort through them using its own judgment. The same judgment that causes hallucinations in the first place. This dissertation introduces Dynamic Context-Aware Trie (DCA-Trie), a method that adjusts valid knowledge graph paths during decoding rather than fixing them at the start. A sentence transformer scores candidate paths against both the original question and the output generated so far, letting the constraints shift as reasoning progresses rather than holding still from the beginning. One variant applies this filtering when the trie is first built; another updates the constraints step by step during generation. To measure how loosely or tightly a constraint operates (independently of whether the final answer is correct) this work introduces the Semantic Irrelevance Ratio (SIR).

DEDICATION

Dedicated to our parents. For every sacrifice we didn't see and every bit of help we couldn't have asked for. Thank you for everything.

ACKNOWLEDGEMENTS

We thank Dr. Eric Affum for his guidance, patience, and support throughout this project. His supervision was instrumental in shaping the direction of this work. We also express our appreciation to our families, friends, and mentors for their encouragement and understanding throughout the process. Finally, we acknowledge the academic community whose work has informed and inspired this research. This dissertation builds on the contributions of many scholars, and we are grateful for their work.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
BFS	Breadth-First Search
CoT	Chain-of-Thought
CWQ	ComplexWebQuestions
DCA-Trie	Dynamic Context-Aware Trie
DoG	Decoding on Graphs
FNR	False Negative Rate
FSM	Finite State Machine
GCD	Grammar-Constrained Decoding
GCR	Graph-Constrained Reasoning
GNN	Graph Neural Network
GPU	Graphics Processing Unit
KG	Knowledge Graph
KGQA	Knowledge Graph Question Answering
LLM	Large Language Model
NLP	Natural Language Processing
QA	Question Answering
RAG	Retrieval-Augmented Generation
RLHF	Reinforcement Learning from Human Feedback
SBERT	Sentence Bidirectional Encoder Representations from Transformers
SIR	Semantic Irrelevance Ratio
SQL	Structured Query Language
VRAM	Video Random Access Memory

CHAPTER 1

INTRODUCTION

1.1 Background

Graph-constrained decoding promises to eliminate hallucination in knowledge graph question answering by restricting the LLM to tokens that correspond to real graph facts. This promise rests on a structural guarantee: every generated token follows a valid KG path. But the guarantee is bought at a cost that the field has not yet confronted. The constraint oracle is built once, before generation begins, from graph topology and question entities alone. It does not know what the question asks. It does not update as reasoning unfolds. The result is a system that is structurally faithful but semantically permissive, admitting thousands of irrelevant paths at every step and forcing the model to choose among them using the same parametric knowledge that constrained decoding was designed to eliminate.

This is the tension that motivates the present work. The background below traces how it arises.

Large language models (LLMs) have grown from narrow text predictors into systems capable of multi-step reasoning, summarisation, and broad question answering (Brown *et al.*, 2020; OpenAI, 2024). The GPT series and the Llama family achieve these capabilities by learning statistical patterns from massive text corpora, internalising factual associations as implicit weight regularities rather than as verifiable entries (Touvron, Martin, and Stone, 2023; Dubey *et al.*, 2024). The same process that makes them broadly capable makes them structurally unreliable on knowledge-intensive tasks: because knowledge is stored as weights, a model can produce fluent, confident answers that are factually wrong.

Hallucination (the generation of fluent but incorrect content) occurs at rates from 15% to over 60% across a range of tasks (Ji *et al.*, 2023). Huang *et al.* (2025) distinguish intrinsic hallucinations (contradicting the source) from extrinsic ones (unverifiable by any source). The mechanism is autoregressive: each token is selected from a distribution conditioned only on previously generated tokens, with no checkpoint against external knowledge. An error at

step t propagates coherently through the rest of the output (Huang *et al.*, 2025; Ji *et al.*, 2023). Recent theoretical work suggests this is inherent to statistical language modelling, not merely a consequence of insufficient training (Xu, Jain, and Kankanhalli, 2025). Scaling does not fix it. Larger models hallucinate differently, not less.

Knowledge graph question answering (KGQA) makes this problem concrete. A KG stores facts as *(subject, relation, object)* triplets, for example *(Paris, capital of, France)*, enabling reasoning through interconnected relationships (Bollacker *et al.*, 2008). Benchmarks such as WebQSP (Yih *et al.*, 2016) and ComplexWebQuestions (Talmor and Berant, 2018) are built on Freebase and frequently require multi-hop reasoning across several relations. An unconstrained LLM can still produce a fluent but incorrect answer because it relies on parametric memory instead of verifying each step (Lan *et al.*, 2022a; He *et al.*, 2021).

Retrieval-augmented generation (RAG) retrieves relevant passages and adds them to the prompt (Lewis *et al.*, 2020; Izacard and Grave, 2021). It reduces hallucination, but the retrieved information only influences generation rather than constraining it. The model may ignore retrieved evidence or generate beyond it (Asai *et al.*, 2024; Agrawal *et al.*, 2024).

Constrained decoding takes a harder line. A constraint oracle computes a binary mask at each step, setting invalid tokens to $-\infty$ before the softmax (Geng *et al.*, 2023; Willard and Louf, 2023). Graph-Constrained Reasoning (GCR) (Luo *et al.*, 2025a) applies this to KGQA by pre-building a KG-Trie that encodes all valid graph paths within L hops of the question entities. GCR achieves 92.6 Hits@1 on WebQSP with 100% structural faithfulness: every generated token corresponds to a real graph fact.

But the trie is static. It is constructed once, before generation, from graph topology and question entities only. It does not consider what the question means or how far reasoning has progressed. The valid token set at the first generation step is identical to the valid token set at the tenth. On multi-hop questions this makes the oracle excessively permissive: thousands of structurally valid but semantically irrelevant paths remain in the constraint set at every step, and the model must filter among them using parametric knowledge, precisely the behaviour constrained decoding was designed to eliminate (Li *et al.*, 2025; Luo *et al.*, 2025a).

Decoding on Graphs (DoG) (Li *et al.*, 2025) updates the trie step-by-step as generation

proceeds, but expansion is guided by graph topology alone. Question semantics play no role in determining which paths remain valid. The model must still discriminate between relevant and irrelevant candidates using parametric knowledge, while incurring the additional cost of rebuilding the constraint structure during inference.

Neither method narrows the constraint set in response to what the question is asking. DCA-Trie addresses this by conditioning the valid token set on the knowledge graph, the input question, and the partial reasoning chain. The goal is structural faithfulness without semantic permissiveness: every admitted token must correspond to a real graph fact, but the set of admitted facts should shrink as reasoning progresses.

1.2 Problem Statement

Existing KG-constrained decoding frameworks define valid tokens as $W_{\text{val}} = f(G, E_q)$, a function of the graph and question entities only (Luo *et al.*, 2025a; Li *et al.*, 2025). This preserves structural faithfulness but not contextual relevance: the oracle admits every reachable path, whether or not it points in the direction the question requires.

The gap widens in multi-hop settings. The number of candidate paths grows rapidly with each hop (Lan *et al.*, 2022a; Talmor and Berant, 2018), and the LLM must filter among structurally valid but semantically irrelevant options using parametric knowledge. This reintroduces the internal-memory reliance that constrained decoding was designed to eliminate.

Recent work such as RouterKGQA (Yuan *et al.*, 2026) applies semantic filtering at answer selection, after generation. The model remains exposed to irrelevant candidates throughout the generation process. DCA-Trie filters during decoding, at the token-validity stage, preventing the model from considering irrelevant reasoning tokens in the first place.

This project extends the validity function from $W_{\text{val}}(t) = f(G, E_q)$ to:

$$W_{\text{val}}(t) = f(G, E_q, q, y^{<t}) \tag{1.1}$$

where q is the input question and $y^{<t}$ is the partial generation. The oracle must satisfy three requirements. First, it must preserve structural faithfulness: every accepted token matches a valid KG path. Second, it must reduce semantic irrelevance, measured by the Semantic

Irrelevance Ratio (SIR; Chapter 3). Third, it must maintain answer recall: false negatives on gold paths stay below 5% on WebQSP validation.

1.3 Objectives of Thesis

Aim

To design, implement, and evaluate a Dynamic Context-Aware Trie (DCA-Trie) framework for knowledge graph-constrained LLM generation, conditioning each decoding step on the knowledge graph, the input question, and the partial reasoning chain to improve multi-hop KGQA accuracy. The framework must preserve 100% structural faithfulness throughout.

Objectives

To achieve this aim, the following objectives have been established:

i. To characterize the constraint permissiveness problem in existing KG-constrained decoding frameworks by defining and measuring the Semantic Irrelevance Ratio (SIR) of GCR's static KG-Tries on WebQSP, stratified by hop depth.
ii. To design a symbolic constraint oracle (TypeOracle) that filters candidate KG paths using two ontology-based gates, a range gate checking property ranges at each hop and a type gate checking terminal entity types at the final hop, eliminating the encoder cost and threshold sensitivity of embedding-based approaches.
iii. To implement DCA-Trie v1, a static filtering variant that applies the TypeOracle at KG-Trie construction time, reducing trie size and keeping the false negative rate below 5% on gold paths on the WebQSP validation set.
iv. To implement DCA-Trie v2, a dynamic variant in which the trie is updated after each committed triplet using the TypeOracle conditioned on the question and partial generation $y^{<t}$, directly operationalizing Eq. 1.1.
v. To evaluate DCA-Trie v1 and v2 against the GCR baseline and an unconstrained chain-of-thought (CoT) baseline on WebQSP, measuring Hits@1, F1, structural faithfulness, SIR, and average trie size, with results stratified by question hop depth.
vi. To demonstrate practical applicability through an interactive prototype, implemented as a command-line QA interface, allowing users to enter natural language questions

and receive fact-based reasoning chains with answers.

1.4 Tools and Facilities Used

1.4.1 Software Tools

Table 1.1 presents the software tools used in this project.

Table 1.1 Implemented Software Tools Evaluation.

Tool	Use in Project	Rationale
Python / HuggingFace Transformers	Model inference pipeline; loading and running GCR's Llama-3.1-8B checkpoint; integrating the relevance scorer	Provides reliable model loading, tokenization, and generation utilities, and enables direct use of the released GCR checkpoint.
GCR Framework	Primary baseline; KG-Trie construction, graph-constrained decoding, and result reproduction on WebQSP and CWQ	Provides a strong baseline and supports the trie-based constrained decoding mechanism that DCA-Trie extends.
Sentence-transformers / all-MiniLM-L6-v2	Semantic relevance scoring; computing cosine similarity between question-context and candidate KG path embeddings	Provides fast semantic embeddings without extra fine-tuning and supports efficient iterative decoding.
PyTorch	Deep learning framework underpinning all model inference and custom decoding logic	Supports implementation and testing of custom decoding logic and integrates smoothly with HuggingFace.
Freebase / WebQSP and CWQ Datasets	Knowledge graph source and KGQA benchmarks for evaluation	Provides standard datasets for this research area and enables direct comparison with prior work.
Google Colab Pro (A100, 40 GB)	GPU environment for all inference experiments; Llama-3.1-8B constrained decoding with beam search	Provides enough GPU memory for stable experiments and removes the need for local high-end hardware.
Visual Studio Code + Git / GitHub	Development environment and version control for all four team members	Supports day-to-day coding, team collaboration, version tracking, and reproducibility throughout the project.

1.4.2 Facilities

The following facilities were utilized in the execution of this project:

i. Google Colab Pro (A100, 40 GB VRAM) for all model inference and constrained decoding experiments.

ii. University library and online academic databases (ACM Digital Library, arXiv, Google Scholar, ACL Anthology) for literature review and citation verification.

iii. University internet infrastructure for cloud connectivity, dataset access, and retrieval of model checkpoints from the HuggingFace model hub.

iv. Shared team repository hosted on GitHub for version control, experiment logging, and reproducibility of all reported results.

1.5 Methods Used

The project employs the following methods:

i. Systematic review and critical analysis of the academic literature on constrained decoding, knowledge graph question answering, hallucination in LLMs, and retrieval-augmented generation.

ii. Formal gap analysis of the four foundational papers (GCR (Luo *et al.*, 2025a), DoG (Li *et al.*, 2025), GCD (Geng *et al.*, 2023), and Outlines (Willard and Louf, 2023)) to identify the permissiveness problem and the absence of generation-state feedback in the constraint oracle.

iii. Formal specification of the DCA-Trie framework, including the SIR metric, the relevance scoring mechanism, the static filtering procedure for v1, and the step-wise dynamic expansion rule for v2.

iv. Reproduction of the GCR baseline on WebQSP using GCR's publicly released Llama-3.1-8B checkpoint to establish a verified starting point within 1–2% of published Hits@1.

v. Quantitative evaluation of all configurations on WebQSP and CWQ using Hits@1, F1, faithfulness rate, SIR, and average trie size per step, stratified by question hop depth following the protocol of Lan *et al.* (2022a).

1.6 Scope of Work

This project targets the permissiveness problem in GCR and DoG: the fact that existing constraint oracles admit every structurally reachable path, regardless of whether the path is relevant to the question being asked. The evaluation is restricted to multi-hop KGQA over Freebase-grounded benchmarks (WebQSP and CWQ), the standard settings in this literature. All experiments use GCR's publicly released Llama-3.1-8B checkpoint (Luo *et al.*, 2025a); no model training or fine-tuning is performed. The semantic relevance scorer uses all-MiniLM-L6-v2 without domain-specific adaptation.

Generalization to knowledge graphs other than Freebase, or to non-KGQA tasks such as entity linking, would answer a different question: whether DCA-Trie transfers across domains. This is identified as future work. Formal proofs of the faithfulness guarantee under dynamic pruning are beyond scope; empirical measurement of the gold answer false negative rate provides the relevant evidence. Production-scale deployment and latency optimization are engineering extensions, not research contributions.

The primary contribution is the DCA-Trie framework and the Semantic Irrelevance Ratio metric. Secondary contributions are the empirical characterization of constraint permissiveness in existing frameworks and the ablation analysis isolating semantic filtering from dynamic expansion. All code, configurations, and curated evaluation datasets will be made publicly available.

1.7 Organization of Work

Chapter 1 identifies the problem and motivates DCA-Trie. Chapter 2 reviews literature on hallucination, KGQA, constrained decoding, and related frameworks, establishing the gaps that motivate context-aware constraint mechanisms. Chapter 3 presents the DCA-Trie formulation, research questions, evaluation criteria, and implementation of v1 and v2 within the GCR pipeline (Luo *et al.*, 2025a). Chapter 4 reports experimental results on WebQSP and CWQ, analyzing structural faithfulness, semantic irrelevance, answer accuracy, and findings from ablation and error analysis. Chapter 5 summarizes contributions, discusses practical implications, and identifies directions for future work.

CHAPTER 2

LITERATURE REVIEW

While large language models (LLMs) are good at many tasks, their use of autoregressive generation makes them likely to produce fluent but incorrect information (Ji *et al.*, 2023). For knowledge-intensive question answering, this unreliability is a major roadblock: the ability to verify reasoning against external facts is essential. To tackle this issue, recent studies have focused on grounding models in structured external facts through knowledge graphs. Still, keeping a model strictly aligned with these graphs during generation, without losing its natural flow, is a difficult challenge.

This literature review looks at the current research on these challenges and examines how knowledge graph constraints might help ensure factual accuracy during multi-hop reasoning. Instead of immediately discussing the proposed solution, this chapter outlines the connections between language model hallucination, multi-step validation, and structured grounding. By assessing current strategies to reduce errors, from improved prompting to fixed decoding, this chapter shows where soft-grounding methods do not meet expectations, highlighting the specific technical gaps this project addresses.

2.1 Large Language Models

To understand how knowledge graph constraints can be used with language models, it is first necessary to grasp how these models generate language. Large language models (LLMs) are the main systems powering today's natural language processing. This section describes the structure and generation features of LLMs. It focuses on the autoregressive process that leads to their smooth reasoning abilities and their risk of producing false information.

2.1.1 Transformer Architecture

Modern LLMs are built upon the Transformer architecture, introduced by Vaswani *et al.* (2017). At its core, the Transformer relies on a scaled dot-product self-attention mechanism that allows the model to dynamically weigh the importance of different tokens in an input sequence regardless of their positional distance. Formally, for a set of queries Q, keys K, and

values V, the attention output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.1}$$

where d_k represents the dimensionality of the keys. This scaled dot-product mechanism in Equation (2.1) marked a significant change from earlier recurrent architectures. It allowed for highly parallel training and the capturing of long-range dependencies. Initially, the framework suggested an encoder-decoder setup for translation tasks. However, the field has mostly moved toward decoder-only models for text generation. In a decoder-only model, self-attention is causally masked. This means that when the model computes the representation for a specific token, it can only focus on previous tokens and not future ones. This strictly left-to-right processing fits well with the task of predicting the next word in a sequence. The Llama-3.1-8B model (Dubey *et al.*, 2024) is a prominent example of this decoder-only, causally-masked transformer design, and has been adopted as the backbone in recent KG-constrained decoding work (Luo *et al.*, 2025a).

2.1.2 Autoregressive Generation

The text production process in these models is fundamentally autoregressive. Given a vocabulary $\mathcal{V}$, an input sequence x, and previously generated output tokens $y_{<t}$, the model computes a probability distribution over the entire vocabulary for the next token: $P(y_t|y_{<t}, x)$. Since the generation condition includes all previously generated tokens, each new word becomes part of the context for the next word. Decoding strategies, such as beam search and nucleus sampling (Holtzman *et al.*, 2020), determine how a specific token is chosen from this calculated distribution. Importantly, because this distribution is always computed over the full vocabulary, any mathematically possible token has a positive, non-zero chance of being generated at any step. This autoregressive nature is what creates fluent, coherent text. However, since each step relies on prior outputs without an external verifier, it also directly causes hallucination.

2.1.3 Large-Scale Pretraining

Large language models emerged through a fundamental shift in scale: current models are trained on vastly larger datasets using billions more parameters than earlier transformers.

Foundational models are pretrained on vast amounts of unlabelled text using a simple self-supervised goal, usually predicting the next token. The GPT series (Brown *et al.* 2020; OpenAI 2024) and the open-weight Llama series (Touvron, Martin, and Stone 2023; Dubey *et al.* 2024) show that scaling this simple task leads to powerful downstream capabilities, including few-shot learning and broad world knowledge. During pretraining, the model internalizes patterns, facts, and logical structures found in its training data. However, this memory is fundamentally static and statistical. While increasing the size of models improves their fluency, syntax, and ability to recall common facts, it does not address the underlying generation process. The model still lacks a way to interact with dynamic external facts or ensure that its statistical outputs reflect verified truths in the real world.

2.1.4 Instruction Following and Chain-of-Thought

To better align these pretrained base models with user intent, they go through post-training processes like instruction tuning and reinforcement learning from human feedback (RLHF). This alignment lets models engage in dialogue and follow complex instructions. One of the key abilities of aligned LLMs is multi-step reasoning, often prompted through Chain-of-Thought (CoT; Bosma *et al.* 2022). As surveyed by Huang and Chang (2023), CoT prompting encourages the model to produce intermediate reasoning steps before reaching a final answer, effectively breaking down complex problems. While CoT greatly enhances performance on logic and reasoning tasks, it suffers from the same autoregressive limitation: each intermediate reasoning step is generated using only the model's internal statistical distributions. So, while CoT helps structure a language model's reasoning, it cannot logically connect those outputs to verifiable sources. This ongoing lack of factual grounding shows the need for stricter interventions, such as knowledge graph-constrained decoding.

2.2 Hallucination in Large Language Models

2.2.1 Definition and Taxonomy

Hallucination in natural language generation refers to the production of content that is fluent and internally coherent yet factually unsupported by or in outright contradiction with verifiable sources (Ji *et al.*, 2023). Huang *et al.* (2023) refine this definition into a two-part taxonomy. *Intrinsic hallucination* happens when the model output directly contradicts material present in

the provided source context. *Extrinsic hallucination* occurs when the output contains claims that cannot be verified against any external source, even if it does not directly contradict a specific reference.

Both types are documented at non-trivial rates in deployed systems. Ji *et al.* (2023) survey over 30 works across summarization, dialogue, and question answering, finding hallucination rates ranging from 15% to over 60% depending on task type and measurement methodology. In the specific context of knowledge graph question answering, Luo *et al.* (2025a) report that retrieval-augmented reasoning approaches produce hallucinated reasoning steps in approximately one-third of cases even when provided direct access to a supporting knowledge graph, underscoring that the problem is not simply one of insufficient external knowledge.

2.2.2 Structural Causes: The Autoregressive Generation Mechanism

The primary cause of hallucination is the autoregressive generation process found in all modern large language models. At each generation step, the model computes a probability distribution over its vocabulary based on the input and all previously generated tokens. Three structural aspects of this mechanism make hallucination an ongoing risk rather than just a rare failure.

First, the generation distribution is based entirely on learned parameters, with no part that corresponds to factual verification against an external source. The model has no mechanism to "check" a candidate token against a knowledge base before selecting it. Second, errors propagate forward: because each step is conditioned on all prior output, a hallucinated token at step t becomes part of the conditioning context for step $t+1$, producing fluent and internally consistent continuations of an already-incorrect chain. Third, the distribution is defined over the full vocabulary at every step, meaning that any token has a positive chance of being selected at any point, regardless of factual correctness (Brown *et al.*, 2020; Huang *et al.*, 2023; Ji *et al.*, 2023).

2.2.3 The Scaling Paradox

A common argument against the severity of the hallucination issue is that large models should eventually achieve enough factual reliability. However, evidence does not support this view. Lin, Hilton, and Evans (2022) introduce TruthfulQA, a benchmark for questions that often

produce plausible but false responses. Their main finding is that larger models do not perform better than smaller models in terms of truthfulness and in some categories show *inverse scaling*. In other words, larger models can do worse because they reproduce widespread but incorrect beliefs from their training data more effectively.

Perez *et al.* (2023) identify a compounding issue in models trained with reinforcement learning from human feedback: these models learn to generate confident-sounding responses that align with apparent user expectations rather than verifiable facts, separating confidence from accuracy. Kandpal *et al.* (2023) show that while larger models memorize more training-data facts and improve on frequently-seen entity combinations, they do not improve on compositional questions requiring combinations of facts not seen during training. This precisely mirrors the structure of multi-hop knowledge graph questions. Mallen *et al.* (2023) confirm significant hallucination rates for GPT-4 on questions involving tail entities, regardless of model size. Together, these findings show that hallucination is a structural feature of the autoregressive mechanism, not a scaling issue with a simple solution.

2.3 Multi-Step Reasoning: Chain-of-Thought and Its Limits

2.3.1 Chain-of-Thought Prompting

Chain-of-thought (CoT) prompting, introduced by Bosma *et al.* (2022), is the dominant paradigm for eliciting multi-step reasoning from large language models. By including worked examples in the prompt that show intermediate reasoning steps, the model learns to decompose complex problems into a sequence of simpler sub-problems before producing a final answer. Bosma *et al.* (2022) demonstrate substantial accuracy improvements on arithmetic, commonsense, and multi-hop question answering benchmarks, with the capability emerging at approximately 100 billion parameters. Zero-shot chain-of-thought (Kojima *et al.*, 2022) achieves similar benefits without worked examples by adding the phrase "Let's think step by step" to the prompt, demonstrating that the underlying capability is latent in sufficiently large models and requires only elicitation.

Despite these improvements, CoT has a fundamental structural limitation: each intermediate step is generated by the same autoregressive mechanism described in the previous section. An incorrect intermediate step propagates forward as conditioning context, and no mechanism

verifies that any step corresponds to a fact in an external knowledge source. A CoT chain can be entirely fluent and internally coherent while containing one or more steps that are factually fabricated.

2.3.2 Extensions: Self-Consistency and Tree-of-Thought

Two major extensions of chain-of-thought tackle its reliability issues without fixing its structural faithfulness problem. Self-consistency (Wang *et al.*, 2023c) samples several independent CoT chains and chooses the final answer based on majority vote, minimizing individual chain errors. This method significantly improves accuracy on benchmarks like GSM8K and multi-hop QA by reducing sensitivity to individual hallucinated chains. However, it does not guarantee that any single chain is factually accurate. A majority of independently hallucinated chains can still produce an incorrect majority-vote answer, especially on deep multi-hop questions where error rates increase.

Tree-of-Thought (Cao *et al.*, 2023) treats generation as a search through a tree of partial reasoning sequences. It uses the language model both as a generator and as an evaluator to trim unpromising branches. This approach enhances performance on tasks that need backtracking and exploration. However, the key limitation persists: pruning decisions are made by the model itself, relying on its own parameters as the only evaluator, without any outside verification of the factual accuracy of candidate reasoning steps.

2.3.3 The Faithfulness Ceiling of Prompting-Based Approaches

The main limitation of all prompting-based methods, such as few-shot, chain-of-thought, self-consistency, tree-of-thought, and their combinations, is that they modify the *input* to the language model without modifying the *generation mechanism*. Since decoding happens over the full vocabulary, each token has a strictly positive probability at every step. True structural faithfulness means ensuring that each generated token matches a verified external fact. To achieve this, invalid tokens must be given exactly zero probability. This can only happen by directly changing the decoding algorithm. This is the purpose of constrained decoding methods (Geng *et al.*, 2023; Willard and Louf, 2023).

2.4 Knowledge Graphs and Knowledge Graph Question Answering

2.4.1 Knowledge Graphs

A knowledge graph is a structured database of verified facts, represented as a set of directed, labelled triplets (e, r, e') where $e, e' \in \mathcal{E}$ are entities and $r \in \mathcal{R}$ is a relation type. Each triplet encodes a single human-verified fact: for example, (*Marie Curie*, *award_received*, *Nobel_Prize_Physics*) or (*Blue_Hawaii*, *film.film.featured_film_location*, *Hawaii*). Figure 2.1 illustrates a simplified fragment of such a structure, demonstrating how these triplets interlink to form a broader network. Freebase (Bollacker *et al.*, 2008), the knowledge graph underlying the evaluation benchmarks used in this work, encodes tens of millions of such triplets across geography, history, entertainment, and science, and has been widely adopted as the standard resource for KGQA research (Yih *et al.*, 2016; Talmor and Berant, 2018).

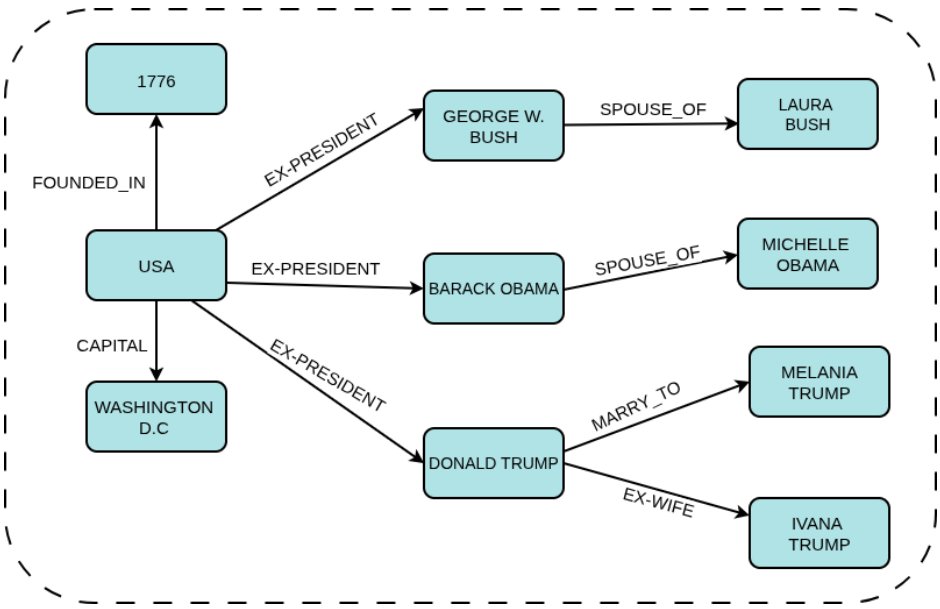

Figure 2.1 A simplified example of a knowledge graph fragment.

The critical distinction between a knowledge graph and an unstructured text corpus is that every triplet in the KG has been explicitly verified and is machine-readable in a deterministic, unambiguous form. This property makes KGs suitable as sources for hard constraints on generation: a candidate token can be verified against the KG via an exact lookup, rather than via probabilistic semantic matching against raw text. A multi-hop reasoning path of length L through the KG is a sequence $(e_0, r_1, e_1, \ldots, r_L, e_L)$ in which each consecutive triplet (e_{i-1}, r_i, e_i) is a verified member of $\mathcal{T}$ (Lan *et al.*, 2022b).

2.4.2 Knowledge Graph Question Answering

Knowledge graph question answering (KGQA) requires a system to identify the answer entity e_L to a natural language question q by reasoning through a chain of verified KG triplets. The task is formally defined as: given question q with associated question entities $\mathcal{E}_q \subseteq \mathcal{E}$ identified by entity linking, identify the terminal entity of the correct reasoning path in $\mathcal{G}$ (Lan *et al.*, 2022b).

Early KGQA approaches relied on either semantic parsing (converting questions to structured SPARQL queries) or information retrieval over KG subgraphs (Lan *et al.*, 2022b). More recent approaches use large language models as retrievers, agents iteratively querying the KG, or generators conditioned on retrieved subgraph context (Jiang *et al.*, 2023a; Jin *et al.*, 2024; Luo *et al.*, 2024). The move toward LLM-based KGQA has improved fluency and coverage, but it has also brought back the risk of hallucination that structured methods avoided. LLM-generated reasoning chains are not guaranteed to match confirmed KG paths.

2.4.3 Evaluation Benchmarks

Two benchmark datasets provide the evaluation framework for KGQA research. WebQSP (Yih *et al.*, 2016) comes from the WebQuestions dataset. It contains 4,737 questions based on Freebase, with reasoning depths of one to two hops. These questions include SPARQL queries and are evaluated using Hits@1 and F1. ComplexWebQuestions (CWQ; Talmor and Berant 2018) builds on WebQSP by transforming questions through conjunction, composition, and comparative operations, resulting in about 34,000 questions that require up to four reasoning hops. WebQSP illustrates a case where the valid paths are relatively few. In contrast, CWQ shows a case with many valid paths, making it harder to find the correct path among structurally valid options.

2.4.4 Multi-Hop Complexity

The complexity of multi-hop KGQA rises sharply with hop depth. For an entity with an average out-degree d in the KG, the number of valid paths of length L from the question entities increases as $|\mathcal{E}_q| \times d^L$. For well-connected Freebase entities where d is around 20 and a question may have up to three question entities, a three-hop question can generate up to 24,000 structurally valid paths, but only one leads to the correct answer (He *et al.*, 2021;

Lan *et al.*, 2022b). This rapid growth has real implications for any method that needs to distinguish between candidate paths. At each step, the model must choose the correct next token from a vast array of valid yet incorrect options, relying on its parametric knowledge to do so.

2.5 Constrained Decoding for Faithful Text Generation

Constrained decoding restricts generation to a predefined valid set at each step by directly intervening in the decoding mechanism. Unlike prompting, which changes the input to the LLM, constrained decoding alters the token selection process itself. Invalid tokens receive zero probability before sampling, making their generation structurally impossible.

2.5.1 Logit Masking

The main mechanism of constrained decoding is logit masking, formally described in Geng *et al.* (2023) and Willard and Louf (2023). This technique works by computing a binary mask over the vocabulary at each generation step, setting the log-probability of invalid tokens to $-\infty$ before the softmax, thereby forcing their selection probability to exactly zero. This ensures that invalid tokens are structurally impossible to generate, rather than merely discouraged. The following subsections examine two concrete instantiations of logit masking: first for enforcing syntactic and formal correctness, then for enforcing knowledge graph path validity.

2.5.2 Grammar-Constrained Decoding

Geng *et al.* (2023) introduce Grammar-Constrained Decoding (GCD), which constrains generation to strings conforming to a context-free grammar, using an Earley parser as the oracle. The valid next-token set at each step is the set of tokens that extend the current partial string as a valid grammar prefix. GCD additionally introduces input-dependent grammars, in which the grammar is parameterized by the input question, representing a partial step toward context-sensitive constraints. Willard and Louf (2023) address the computational cost of constrained decoding with an efficient finite-state machine formulation. By precomputing an index from automaton states to valid vocabulary subsets, the valid token set can be retrieved in amortized constant time at each generation step, making constrained decoding practical at inference speed.

The shared limitation of these format-constraint approaches is that they enforce structural validity of the output format (such as grammatical correctness or schema conformance) without enforcing factual correctness. A grammatically valid output can contain entirely hallucinated facts. The transition from format constraints to factual constraints requires a constraint oracle that verifies tokens against a knowledge source rather than against a grammar.

2.5.3 Knowledge Graph-Constrained Decoding

Applying constrained decoding to knowledge graph faithfulness is a more recent development. The central idea is to use the KG itself as the constraint oracle. At each generation step, only tokens that correspond to valid continuations of a KG reasoning path are allowed. This method guarantees structural faithfulness, meaning every generated token matches a verified KG triplet, making hallucination of KG facts structurally impossible.

Graph-Constrained Reasoning (GCR) (Luo *et al.*, 2025a) is the leading work in this area. GCR builds a KG-Trie (a prefix tree of all KG paths within L hops of the question entities, created by breadth-first search before generation begins) and uses it as the constraint oracle during beam-search decoding. The trie is queried at each step to provide valid next tokens. GCR achieves 92.6 Hits@1 on WebQSP and 75.8 on CWQ with verified 100% structural faithfulness, using a Llama-3.1-8B backbone. One important feature is that the trie is created once before generation and remains unchanged: the valid token set is fixed no matter what has been generated.

Decoding on Graphs (DoG) (Li *et al.*, 2025) presents step-wise trie expansion as a partial solution to GCR's static oracle. DoG defines *well-formed chains* as sequences of KG triplets where each triplet exists in the KG, and each entity is connected to the previous triplet or is a question entity. It enforces this rule through step-wise expansion. After locking in an entity, the constraint trie is expanded with all KG neighbors of that entity. This makes the oracle topologically dynamic, growing as reasoning moves through the graph. DoG achieves 90.99 Hits@1 on WebQSP and 76.08 on CWQ with 100% structural faithfulness, and it is training-free, needing no fine-tuning. However, the expansion rule includes all KG neighbors of each visited entity, regardless of their relevance to the question or current reasoning direction. Additionally, constructing the trie step-by-step compromises the efficiency of the precomputed constraint structures.

RouterKGQA (Yuan *et al.*, 2026) takes a different approach by routing between a specialized KG-reasoning model and a general LLM based on question complexity. It uses semantic filtering with SBERT embeddings (specifically `nomic-embed-text-v1`) to select candidate answers after generation, achieving competitive accuracy while lowering inference costs. RouterKGQA's semantic scoring operates at the answer selection level, rather than at the token constraint level. Paths are created first and filtered afterward, instead of being constrained during generation.

2.5.4 Beam Search and Trie-Based Constraints

To understand how knowledge graph constraints are applied during generation, it is important to look at the two main search algorithms and data structures involved: beam search and the prefix trie.

Beam Search. First introduced in the HARPY speech recognition system (Lowerre, 1976), beam search is a heuristic search algorithm used in natural language processing to decode sequences from probabilistic models. Unlike greedy decoding, which only picks the single most probable token at each step and risks creating sub-optimal global sequences, beam search keeps a set of the k most promising partial hypotheses (or "beams") at all times. At each generation step, the algorithm expands all k current hypotheses by scoring their potential continuations, ranks the resulting sequences based on their total log-probability, and reduces the search space back to the top k candidates (Holtzman *et al.*, 2020). This method reduces the chance of early errors multiplying while still being computationally viable. Thus, it is the primary decoding strategy for tasks that need structured and coherent output.

The Prefix Trie. A trie, originating from the concept of "trie memory" (retrieval) introduced by Fredkin (1960), is a deterministic tree-based data structure used to store a changing set of sequences, usually strings. In a trie, no single node holds a complete string. Instead, a node's position in the tree indicates the exact prefix it represents. The branches from any node show the set of valid next characters or tokens. In constrained decoding, a pre-computed trie acts as a highly efficient constraint oracle. Given the current sequence prefix, navigating the trie to the correct node yields the specific subset of valid next tokens in constant time, $O(1)$, concerning vocabulary size. This operation is significantly quicker than continuous dynamic parser evaluations, making trie lookups the most scalable option for applying firm

factual constraints at inference speed (Willard and Louf, 2023).

The KG-Trie. For knowledge graph-constrained generation, models like Graph-Constrained Reasoning (GCR) use a specific adaptation known as the KG-Trie (Luo *et al.*, 2025a). A KG-Trie serializes structurally verified multi-hop paths from the knowledge graph into string formats and maps them across the language model's precise token vocabulary. Rather than nodes representing single alphabetical characters, each edge in the KG-Trie represents a specific valid token in the LLM's vast vocabulary space.

When beam search is merged with a KG-Trie oracle as visualized in Figure 2.2, the decoding process transforms into a strictly bounded traversal of the prefix tree.

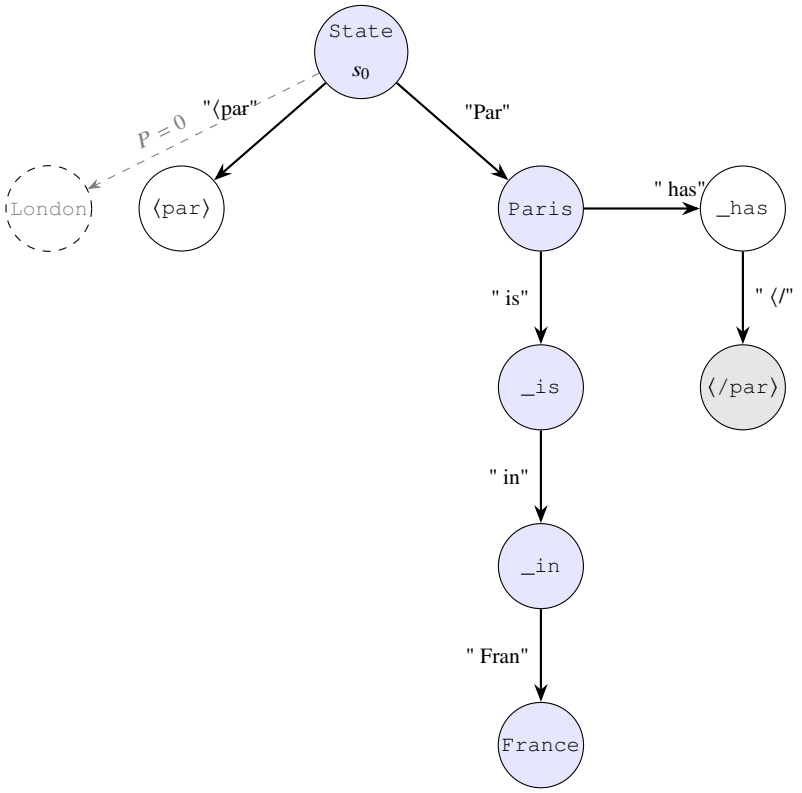

Figure 2.2 Visualization of KG-Trie Constrained Generation.

At each autoregressive step, the current beam hypotheses map to specific states in the KG-Trie. The language model calculates a probability distribution over its entire vocabulary, but a mathematical mask forces the probability of any candidate token not validly present as a child node in the trie to exactly zero ($-\infty$ prior to softmax normalization). The beam search then advances strictly down the permissible branches, functionally restricting the LLM's autoregressive generation to a set of pre-verified deterministic paths while still allowing

the model to express internal preference (via log-probabilities) among those factually valid chains.

2.6 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG), introduced by Lewis *et al.* (2020) in 2020, is the leading approach for grounding LLM outputs in external knowledge. A dense retriever picks relevant passages from an external source based on embedding similarity. A reader LLM then generates the answer based on the retrieved context and the question. Fusion-in-Decoder, proposed by Izacard and Grave (2021) in 2021, enhances this by encoding multiple retrieved passages separately and merging them in the decoder, which improves coverage.

2.6.1 KG-Augmented Retrieval

Several studies enhance RAG with knowledge graph (KG)-structured context. He *et al.* (2021) retrieve KG subgraphs and use attention on retrieved facts to aid multi-hop reasoning. Yasunaga *et al.* (2021) combine retrieved text passages and KG neighborhoods through graph neural networks. Shi *et al.* (2021) retrieve KG paths as natural language context strings, providing path-shaped input to the reader. G-Retriever, developed by He *et al.* (2024) in 2024, applies RAG specifically to textual graph understanding, retrieving graph-structured context for question-answering tasks.

2.6.2 RAG Variants

Recent RAG variants focus on the timing and accuracy of retrieval. Self-RAG (Asai *et al.*, 2024) adds reflection tokens that allow the model to decide when retrieval is necessary and whether retrieved passages support the generated claim. This improves retrieval precision and citation accuracy. FLARE (Jiang *et al.*, 2023b) uses the model's next-token prediction confidence as a trigger for retrieval, obtaining new context when confidence drops below a certain threshold. IRCoT (Trivedi *et al.*, 2023) mixes retrieval with chain-of-thought steps, fetching relevant context at each reasoning step, significantly improving multi-hop accuracy. GraphRAG (Edge *et al.*, 2024) builds community-structured knowledge graphs from document collections and retrieves community summaries for complex relational queries. This approach uses graph structure to boost coverage.

2.6.3 The Structural Limitation of Soft Grounding

Despite their empirical improvements, all RAG approaches share a structural limitation: retrieved context is a soft influence on generation. As shown in Figure 2.3, the retriever determines what the LLM sees; it does not determine what the LLM is permitted to generate. Because the LLM's generation mechanism remains unchanged (the distribution over the full vocabulary is computed autoregressively), there exists a nonzero probability of generating any token at any step, including tokens that correspond to no verified KG fact. Structural faithfulness with probability one, which guarantees that every generated token corresponds to a verified fact, cannot be achieved under any architecture that modifies only the input context without modifying the token selection mechanism (Luo *et al.*, 2025a; Geng *et al.*, 2023).

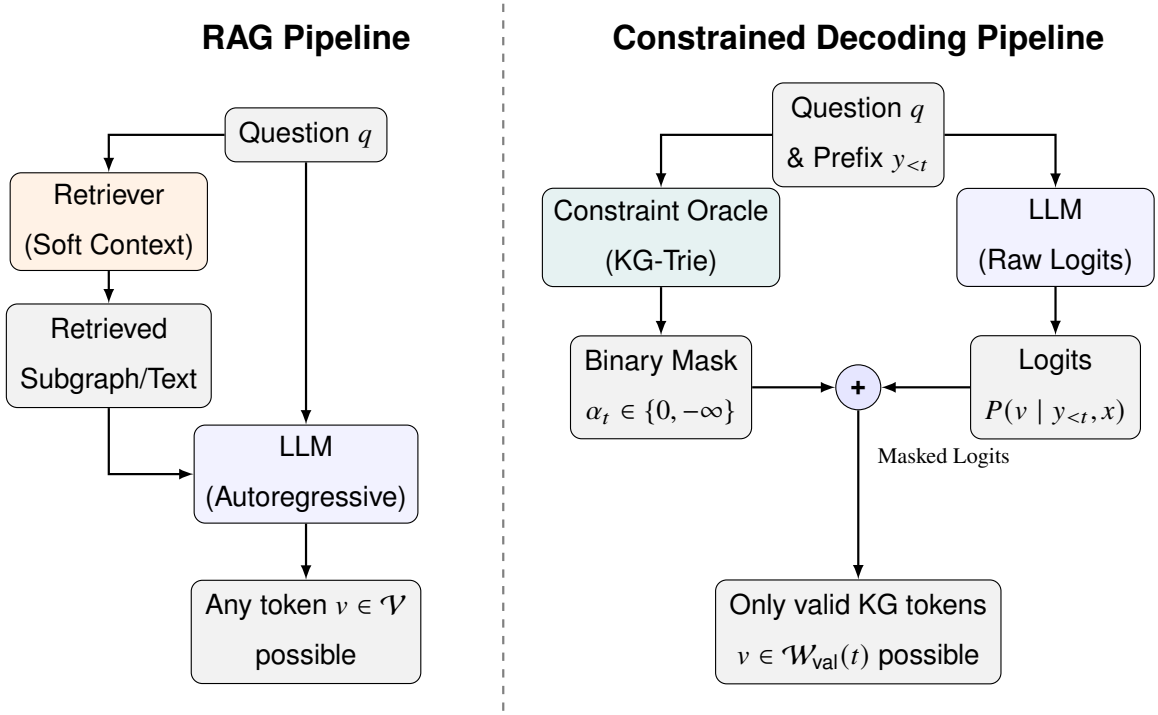

Figure 2.3 Standard RAG vs. KG-Constrained Decoding

2.7 Semantic Relevance Scoring with Sentence Transformers

2.7.1 Sentence Transformer Embeddings

Sentence-BERT (Reimers and Gurevych, 2019) introduced a class of sentence encoders trained to produce dense vector representations in which semantically similar sentences cluster together and semantically dissimilar sentences are pushed apart. The SBERT architecture applies mean pooling over token representations from a BERT-like transformer encoder,

producing a fixed-length embedding. Training uses a siamese network on labelled sentence pairs with a classification objective over entailment, neutral, and contradiction labels. The resulting embedding space supports cosine similarity as a reliable proxy for semantic alignment between sentences, without requiring task-specific fine-tuning at inference time.

2.7.2 Applications in Retrieval and Graph Reasoning

Sentence transformer similarity has been applied extensively in dense retrieval settings. Karpukhin *et al.* (2020) demonstrate that a bi-encoder architecture trained on question-passage pairs produces embeddings that outperform sparse retrieval (BM25) on open-domain QA. Xiong *et al.* (2021) refine bi-encoder training, improving coverage of relevant passages. In the KGQA context, Shi *et al.* (2021) and Saxena, Chakrabarti, and Talukdar (2022) apply semantic similarity to rank candidate KG paths for retrieval, selecting high-scoring paths as soft context for the reader LLM. These approaches use cosine similarity as a soft ranking signal over candidate paths, selecting the top-k for context provision.

2.7.3 Encoder Architecture Trade-offs

Choosing a sentence encoder involves balancing quality and latency, which is particularly important in step-wise constrained decoding, where the encoder is used at each generation step. Larger encoders like all-mpnet-base-v2 (110M parameters) and BGE-M3 (Chen *et al.*, 2024) (568M parameters) perform better on semantic textual similarity benchmarks but significantly increase inference latency. Lexical methods like BM25 work quickly but cannot capture semantic similarity between paraphrase-equivalent paths that lack surface tokens. The all-MiniLM-L6-v2 model (Reimers and Gurevych, 2019), with 22M parameters and about 80ms per-query CPU latency, strikes a good balance between semantic quality and inference speed for iterative applications.

2.8 Related Works

This section reviews the broader field of knowledge-augmented reasoning across two complementary research trajectories: (1) *soft-grounding* approaches, which use external knowledge to inform or filter generation without modifying the decoding mechanism, and (2) *hard-constraint* approaches, which restrict generation to verified facts by modifying the token selection process itself. The survey progresses from early structured methods $\rightarrow$ modern

LLM-based soft approaches → semantic relevance techniques → hard-constraint frameworks → concurrent work, establishing how these lines of research have developed largely in parallel without integration at the constraint-oracle level. This progression illuminates the specific technical gaps that motivate the proposed approach.

2.8.1 Semantic Parsing and Early Structured KGQA

Before large language models became mainstream, traditional methods mainly addressed KGQA through semantic parsing or structured representation learning. Two key approaches characterize this period: parsing questions into executable KG queries like SPARQL and aligning question and KG representations into a shared space for reasoning.

In the first group, early semantic parsing relied on fixed question templates to create queries, which required deep domain knowledge (Berant *et al.*, 2013; Reddy, Lapata, and Steedman, 2014; Bast and Haussmann, 2015). More recent methods use deep learning to automatically generate these logical forms (Yih *et al.*, 2015; Lan and Jiang, 2020). While these symbolic methods offer high interpretability and precise reasoning, they face major challenges in practice. They need expensive ground-truth logical forms for training, which are hard to obtain, and their performance drops sharply when the KG has missing links. Additionally, model-generated queries often cannot be executed due to small syntax or semantic errors with the underlying graph schema, resulting in no answers and lacking fallback options (Yu *et al.*, 2023; Luo *et al.*, 2025a).

To ease the demand for strict logical forms, the second group of methods integrates text and graph representations into a continuous shared space for reasoning. Some methods match question representations to KG facts (Miller *et al.*, 2016; Xu *et al.*, 2019) or KG structural representations (Zhang *et al.*, 2018; Sen, Mavadia, and Saffari, 2023). Notable examples include GraftNet (Sun *et al.*, 2018), PullNet (Sun *et al.*, 2019), and NSM (He *et al.*, 2021), which enhance reasoning through deep graph-based approaches. Incomplete graphs have also been explored, where missing links are inferred using KG embeddings (Saxena, Tripathi, and Talukdar, 2020). While these structured techniques avoid hallucination by operating over confirmed data, their expressiveness is limited compared to the generative flexibility and complex multi-step reasoning that modern LLMs provide.

2.8.2 Retrieval-Based KGQA

Earlier KGQA methods relied on structured retrieval from the knowledge graph instead of LLM generation. These methods prevent LLM hallucination by avoiding freeform generative models and working within the structured limits of the KG. The following works cover the main techniques in this area, including semantic parsing, embedding-based retrieval, graph neural network propagation, and path-level supervision.

Yih *et al.* (2016) created the WebQSP benchmark and introduced a semantic parsing method that converts natural language questions into SPARQL queries to be executed against Freebase. This method offers exact symbolic faithfulness by limiting answer derivation to formally verified SPARQL execution. However, it has a key limitation: the parsing step is fragile. Small changes in question phrasing can lead to incorrect parses, resulting in structurally valid but semantically wrong SPARQL queries. Additionally, this method requires extensive annotation of SPARQL queries for training, which is costly and hard to scale for complex compositional questions.

Talmor and Berant (2018) introduced ComplexWebQuestions and a related compositional semantic parsing method that breaks down complex questions into simpler sub-questions before generating SPARQL. This decomposition technique expands the coverage of semantic parsing for compositional queries, but it does not fix the fragility issue. Errors during decomposition can accumulate across sub-questions, causing the final SPARQL query to be structurally valid but answer a subtly different question than the one asked.

He *et al.* (2021) suggested a multi-hop KGQA method that learns intermediate supervision signals to guide path selection through the KG. They showed that explicit intermediate supervision at each hop significantly enhances performance at deeper reasoning levels. By training the model to predict correct intermediate entities, this approach helps ease the combinatorial difficulties of multi-hop searches. Yet, its limitation is that these intermediate supervision signals come from annotated answer paths, which are expensive to gather and might not be available for all benchmarks or KGs. Furthermore, this method is limited to a fixed maximum hop depth and does not adjust its reasoning depth based on the complexity of the question.

Yasunaga *et al.* (2021) introduced QA-GNN, which combines retrieved text passages and KG neighborhood entities in a joint graph neural network. This network propagates information between passage token nodes and KG entity nodes. By reasoning over both types of information, QA-GNN improves performance on questions that require a combination of textual and structured evidence. However, the joint graph mixes verified KG facts with unverified textual claims, meaning noisy or incorrect text passages can distort entity scores derived from the KG structure. The approach also needs to build a joint graph at inference time for each question, leading to significant preprocessing overhead.

Shi *et al.* (2021) retrieve KG reasoning paths by calculating semantic similarity between the question and natural language descriptions of candidate paths. They provide the top-k highest-scoring paths as context strings to a reader LLM. This retrieval-plus-reading setup separates the structured retrieval step from the natural language generation step. The limitation is that the ranking function relies on soft cosine similarity over surface-form path descriptions, which might give high scores to paths that are lexically similar to the question but semantically incorrect. The reader LLM receives paths as plain text, lacking a way to verify that the provided paths are actual KG edges.

Saxena, Chakrabarti, and Talukdar (2022) used entity and relation embeddings to score and select multi-hop KG paths without using an LLM generator. They learned representations from KG structure with embedding objectives, and computed multi-hop path scores by combining relation embeddings. This method stays entirely within the structured KG and avoids generative hallucination. However, embedding-based path scoring struggles to capture the full semantic complexity of natural language questions. Compositional questions involving negation, comparison, or conjunction are hard to encode accurately as relations, leading to a limit on the types of complex questions it can handle.

Lan *et al.* (2022b) conducted a thorough survey of KGQA methods, analyzing the trade-offs between semantic parsing, embedding-based, and retrieval-based techniques across various benchmarks. They found that retrieval-based approaches often have difficulty with tail entities and rare relation types that are not well-represented in training data. They also noted that no single retrieval method excels across all question types and hop depths. This survey supports the shift toward LLM-based methods as more adaptable alternatives.

Sun *et al.* (2019) developed PullNet, which iteratively gathers relevant KG facts and text passages into a dynamically built evidence graph. It performs inference over this graph to answer multi-hop questions. The iterative method allows PullNet to gather evidence at various depths without setting a hop depth in advance. However, its limitation is that the evidence graph combines KG and text sources without assessing their reliability, and the iterative process introduces latency that increases with question complexity.

Das *et al.* (2018) presented "Go for a Walk and Arrive at the Answer," an end-to-end path-finding approach that trains a policy to navigate the KG from a question entity to the answer entity. Reinforcement learning uses the correctness of answers as a reward to train the policy. This method directly models the reasoning process as graph traversal and yields interpretable paths. Its limitation is that reinforcement learning with sparse correctness rewards is quite unstable during training, needing careful reward shaping. The policy also does not generalize well to question types not adequately represented in the training data.

Zhang *et al.* (2022) proposed subgraph-based reasoning that focuses on extracting a relevant subgraph from the full KG. They perform message passing over this subgraph before selecting the answer entity. By limiting reasoning to a local subgraph, this method significantly reduces both the search space and the cost of inference. The subgraph boundary is determined by how close entities are to the question entity, which creates a ceiling on recall. Answer entities that are topologically farther from the question entity in the KG might be left out of the subgraph, even if they are the correct answer. This issue is closely tied to the hop-depth assumptions built into subgraph extraction methods.

These retrieval-based methods prevent LLM hallucination by working within the structured KG rather than generating freely. However, they share a limitation in expressiveness. They are usually restricted to fixed path structures, fixed hop depths, or fixed retrieval limits and do not take advantage of the flexible multi-step reasoning capabilities of modern LLMs. The shift to LLM-based methods in more recent research improved the naturalness and coverage for complex compositional questions, but this came at the expense of the structural faithfulness guarantees provided by retrieval-based methods.

2.8.3 Knowledge Graph-Enhanced LLM Reasoning

More recent advances focus on the joint use of KGs and LLMs for KGQA, leveraging the sophisticated capabilities of LLMs for enhanced reasoning while mitigating hallucinations and knowledge gaps. A foundational observation is that pure prompting-based approaches (such as Chain-of-Thought (CoT) (Bosma *et al.*, 2022), Plan-and-solve (Wang *et al.*, 2023b), DecomP (Khot *et al.*, 2023), Self-consistency (Wang *et al.*, 2023d), and Tree-of-Thought (ToT) (Cao *et al.*, 2023)) achieve multi-step reasoning but not structural faithfulness to external facts. Fine-tuning approaches extending to reinforcement learning (e.g., OpenAI o1) (Hoffmann *et al.*, 2022) improve reasoning quality but still lack hard guarantees against hallucination. Consequently, the field has shifted toward integrating KG access directly into the generation process.

Agent-based and exploratory approaches. Several methods treat LLMs as agents that iteratively query KGs, such as ReACT (Yao *et al.*, 2023), StructGPT (Jiang *et al.*, 2023a), and ToG (Sun *et al.*, 2024). These methods prompt the LLM to explore relevant facts hop-by-hop using beam search on the KG, combining the LLM's reasoning capacity with the KG's structural grounding.**Path retrieval for intermediate supervision.** Others use explicit retrieval of intermediate facts to guide generation. Methods like KD-CoT (Wang *et al.*, 2023a) and RR (He *et al.*, 2022) retrieve facts to guide generative plans. Jin *et al.* (2024) propose Graph Chain-of-Thought (GraphCoT) to explicitly retrieve graph neighborhood information at every intermediate CoT step, making the connection between reasoning and KG structure explicit.

Specialized encoders and fusion architectures. More sophisticated approaches use graph neural networks and specialized encoders to bridge text and graph modalities. GNN-RAG (Mavromatis and Karypis, 2025) and GFM-RAG (Luo *et al.*, 2025b) adopt graph neural networks to capture relational structure. Luo *et al.* (2024) introduce Reasoning on Graphs (RoG), a planning-retrieval-reasoning framework that retrieves reasoning paths for faithful reasoning. StructLM (Zhuang *et al.*, 2024) fine-tunes models over large structured datasets, while DECAF (Yu *et al.*, 2023) merges semantic parsing and LLM reasoning. These approaches achieve higher empirical performance but often depend on instruction-following or require fine-tuning, creating barriers to practical deployment.

Methods emphasizing complete graph exploration. In contrast to methods that retrieve subgraphs, recent work such as Decoding on Graphs (DoG) (Li *et al.*, 2025) and Tree-of-Traversals (Markowitz *et al.*, 2024) reason over the full accessible KG without filtering, enabling complete coverage at the cost of increased search space.

Shared limitation: KG knowledge as soft guidance. Despite architectural diversity, most reviewed methods treat KG knowledge as a suggestion rather than a strict constraint. For example, UniKGQA (Jiang *et al.*, 2022) unifies retrieval and reasoning through shared representations but introduces entanglement between the two failure modes. Methods using embedding-based approaches (OpenKE (Han *et al.*, 2018)) internalize KG knowledge as parametric memory, losing hard faithfulness guarantees. Subgraph-based methods (Zhang *et al.*, 2022; Wang *et al.*, 2021; Sun *et al.*, 2019; Lin, Yang, and Zhang, 2022; Mavromatis and Karypis, 2022) provide KG context but cannot prevent the LLM from generating beyond it. Even with perfect retrieval, the absence of hard structural constraints means the LLM can still ignore or contradict provided facts during generation.

A direct measurement of this limitation comes from Luo *et al.* (2025a), who find that even the most effective KG-enhanced methods produce hallucinated reasoning steps in approximately one-third of cases. This empirically establishes that soft KG grounding has a systematic performance ceiling.

2.8.4 Semantic Similarity in Graph Reasoning

Several studies use semantic similarity signals to guide reasoning over knowledge graphs. Shi *et al.* (2021) rank candidate KG paths based on the semantic similarity between the question and path descriptions, retrieving high-scoring paths as evidence. Saxena, Chakrabarti, and Talukdar (2022) learn embeddings for KG entities and relations to support similarity-based multi-hop path selection. He *et al.* (2021) use attention-guided traversal over retrieved KG subgraphs, with attention weights indicating how relevant each subgraph region is to the question. PathRetriever (Asai *et al.*, 2020) learns to find reasoning paths from evidence graphs by scoring paths using a recurrent model with their node sequences.

These studies show that semantic similarity can effectively tell apart relevant and irrelevant KG paths in a retrieval setting. The main difference in architecture is that semantic similarity

serves as a soft ranking signal. Candidate paths receive scores, and the highest-scoring ones are presented to the LLM as context or selected among the generated candidates. No existing work treats semantic similarity as a hard binary criterion that determines which tokens the LLM can generate at each step. The difference between soft retrieval signals and hard generation constraints is significant. Soft signals influence what the LLM sees, while hard constraints dictate what the LLM can produce.

While semantic similarity provides soft ranking signals to guide retrieval and filtering, a complementary research direction pursues hard constraints that structurally restrict generation at each decoding step. Rather than choosing among many candidate outputs after generation, hard constraint approaches eliminate invalid options before they are produced. This distinction, between soft guidance and hard structural restriction, is fundamental and leads to qualitatively different performance guarantees and interpretability.

2.8.5 Format-Constrained and Entity-Constrained Generation

A separate line of research tackles factual accuracy by imposing strict limits on what a generative model can produce, regardless of the input context. Unlike retrieval-based approaches that limit the information available to the model, these studies change the generation mechanism itself. The following cluster follows the development of this approach from early lexical constraints to grammar-based and automaton-based frameworks.

De Cao *et al.* (2021) introduced autoregressive entity retrieval (GENRE), which limits generation to a specific set of entity names using a trie created from all known entity surface forms. By restricting beam search to valid trie paths, GENRE ensures that each generated token sequence corresponds to a real entity in a fixed vocabulary.

Geng *et al.* (2023) generalise entity-level trie constraints to arbitrary context-free grammar constraints in Grammar-Constrained Decoding (GCD). GCD uses an Earley parser as the constraint oracle, admitting only tokens that extend the current partial string as a valid grammar prefix. GCD additionally introduces input-dependent grammars parameterised by the input question, allowing the valid token set to depend on question content. GCD establishes that grammar constraints are computationally tractable and empirically improve structured output quality across multiple tasks. The limitation is that a context-free grammar

can enforce syntactic or format correctness but cannot encode factual membership in a knowledge graph: a string may conform perfectly to a grammar specification while containing entirely hallucinated entities or relations.

Willard and Louf (2023) address the computational cost of grammar-constrained decoding with an efficient finite-state machine (FSM) formulation. By precomputing an index from FSM states to valid vocabulary subsets, the valid token set can be retrieved in amortised constant time at each generation step, making constrained decoding practical at inference speed for production systems. This formulation reduces the per-step constraint overhead from the Earley parsing cost (which is cubic in input length) to effectively $O(1)$ amortised lookup. The limitation remains the same as GCD: the FSM encodes structural or format correctness, not factual correctness. An FSM-constrained system can still generate structurally valid outputs that are factually incorrect with respect to any external knowledge source.

Deutsch, Upadhyay, and Roth (2019) propose a general algorithm for constrained sequential inference applicable across any structured prediction task, framing constraint enforcement as inference in a product of finite-state automata. This framework unifies a broad range of constraint types under a common algorithmic schema and demonstrates that hard constraint enforcement is compatible with a variety of sequence models. Its limitation for KGQA is that the constraints it enforces are defined over output string structure rather than over external knowledge: the automata encode properties of the output form, not membership in a KG edge set.

Poesia *et al.* (2022) introduce Synchromesh, a constrained generation system for program synthesis that uses a completion engine to enforce syntactic validity of generated code. By grounding constraint enforcement in a language-specific parser, Synchromesh ensures that all generated programs are syntactically valid. This work establishes that constraint oracles can be derived from formal language specifications beyond context-free grammars. Applied to KGQA, an analogous approach would enforce syntactic validity of SPARQL queries; however, a syntactically valid SPARQL query may still reference entities or relations absent from the target KG, producing an empty result set rather than a faithfully grounded answer.

Scholak, Schucher, and Bahdanau (2021) present PICARD, a method for parsing SQL queries with constrained autoregressive inference. PICARD uses an incremental parser to

reject tokens that would make the partial output unparseable or semantically invalid with respect to a database schema, thereby constraining generation to schema-valid SQL. PICARD demonstrates that schema-grounded constraints substantially reduce invalid query generation in text-to-SQL tasks. The limitation for KG settings is that SQL schema constraints enforce structural consistency with a fixed schema, not factual path traversal through a dynamic relational graph. A schema-valid query may still produce an incorrect answer if the underlying reasoning is unsound.

Anderson *et al.* (2017) introduce Guided Open Vocabulary Image Captioning (GOVIC), an early demonstration of constrained beam search for lexically-constrained generation in image captioning, enforcing inclusion of specified words in the output. While the domain differs from KGQA, this work establishes constrained beam search as a practical mechanism for enforcing lexical constraints during generation and demonstrates the feasibility of hard inclusion constraints. Its limitation is that lexical constraints enforce presence of specific surface forms without enforcing their factual correctness or logical coherence with the rest of the generated output.

Hokamp and Liu (2017) introduce constrained decoding via a grid beam search algorithm that enforces specified lexical constraints in neural machine translation. Grid beam search maintains separate hypothesis sets for each subset of satisfied constraints and merges them to produce outputs satisfying all constraints. This algorithm demonstrates that complex constraint sets can be enforced in polynomial time during beam search. The limitation is that the constraint set must be specified in advance and is fixed throughout generation; there is no mechanism for the constraint oracle to consult an external knowledge source at each step to determine which tokens are valid continuations.

Lu *et al.* (2021) propose NeuroLogic Decoding, a decoding algorithm that enforces propositional logic constraints over lexical items during generation, extending the class of enforceable constraints beyond simple inclusion to disjunctions, conjunctions, and negations. NeuroLogic demonstrates that logically complex constraints can be enforced efficiently during beam search without prohibitive computational overhead. For KGQA applications, the limitation is that propositional logic constraints over lexical items still operate at the surface-form level: a NeuroLogic constraint can require that a specific entity name appears in the output, but cannot verify that this entity is connected to the preceding entity by a verified KG edge.

Zhang *et al.* (2023) analyse the computational tractability of constrained decoding under various constraint classes, establishing theoretical bounds on the complexity of constraint enforcement as a function of grammar type and vocabulary size. Their analysis confirms that finite-state constraints admit efficient enforcement via precomputed indices, while context-sensitive constraints require per-step evaluation that may be prohibitively slow. This theoretical result directly motivates the design choice in KG-constrained systems of representing valid paths as tries (finite-state structures) rather than as arbitrary graph reachability queries.

These works collectively establish that hard constraints on generation are computationally tractable and empirically beneficial across multiple domains. Their shared limitation for KGQA is that they enforce output *format* rather than *factual content*: constraining generation to grammatically valid strings, schema-valid queries, or lexically specified surface forms does not ensure that the generated strings correspond to verified KG paths. The extension from format constraints to KG path constraints requires a constraint oracle that encodes the relational structure of the knowledge graph itself, mapping valid token sequences not to grammar derivations but to traversable edges in a specific, dynamic knowledge base.

2.8.6 Hallucination Mitigation in Knowledge-Intensive Tasks

Beyond the constrained decoding approach, various methods aim to reduce hallucination in knowledge-intensive tasks. Retrieval-augmented methods, such as RAG (Lewis *et al.*, 2020), Self-RAG (Asai *et al.*, 2024), FLARE (Jiang *et al.*, 2023b), IRCoT (Trivedi *et al.*, 2023), and GraphRAG (Edge *et al.*, 2024), address this issue by providing relevant external context. This improves factual coverage without changing the generation mechanism. Wang *et al.* (2023c) enhance consistency by majority-voting across multiple Chain-of-Thought (CoT) paths. Huang *et al.* (2024) investigate whether LLMs can self-correct reasoning errors, finding that self-correction is usually ineffective without external feedback. This reinforces the idea that internal knowledge cannot reliably detect its own mistakes.

Post-hoc verification methods (Nakano *et al.*, 2022; Gao *et al.*, 2023) check generated claims against external sources after generation and either revise or flag unsupported statements. While these methods effectively identify errors, they require a separate verification step and do not prevent hallucination during generation. Agrawal *et al.* (2024) systematically analyze failure modes in RAG systems and find that even when relevant context is provided, LLMs

often produce statements not backed by the retrieved evidence.

2.8.7 Concurrent Work

RouterKGQA (Yuan *et al.*, 2026) is the work most closely related to this research in terms of shared components. RouterKGQA directs KGQA queries between a specialized KG reasoning model and a general LLM based on question complexity. It applies SBERT-based semantic similarity (using the nomic-embed-text-v1 encoder) to filter candidate reasoning paths. It achieves strong performance with lower inference costs, averaging about 1.15 LLM calls per question.

The key difference in architecture lies in when semantic scoring is applied. RouterKGQA scores complete reasoning paths after they are generated, using similarity to filter among candidates afterward. This method does not alter the token selection process during generation. The LLM generates freely, and semantic similarity is applied post-hoc to select among the outputs. In contrast, this thesis uses semantic similarity as a hard admission criterion at the token-constraint level. In this setup, similarity determines which tokens the LLM can generate at each decoding step, rather than just which completed outputs are kept later. This represents a qualitatively different architectural change that RouterKGQA does not address. RouterKGQA also does not provide a metric for constraint permissiveness and does not measure how much its candidate sets contain valid but semantically irrelevant paths.

2.9 Synthesis: Three Unresolved Gaps

The review shows that two different research paths, constrained decoding for structural faithfulness and semantic similarity for relevance-guided reasoning, have developed mainly in parallel without being combined at the constraint-oracle level. Constrained decoding methods (Luo *et al.*, 2025a; Li *et al.*, 2025) achieve structural faithfulness but create their constraint oracles without considering question semantics or the context of generation. This leads to valid token sets that include many irrelevant paths. Semantic similarity methods (Shi *et al.*, 2021; Saxena, Chakrabarti, and Talukdar, 2022; Yuan *et al.*, 2026) improve relevance but act as soft signals that affect retrieval or selection after the fact, without changing the generation process.

Three specific gaps emerge from this review.

Gap 1: The Permissiveness Problem. Current KG-constrained decoding frameworks build their constraint oracles based on graph structure and question entities only. They do not consider question semantics or the current generation state. In the case of multi-hop questions, this results in valid token sets with thousands of structurally correct but semantically irrelevant paths at each generation step. The LLM has to sort through these paths using its own knowledge, which reintroduces the risk of hallucination that constrained decoding aimed to eliminate (Luo *et al.*, 2025a; Li *et al.*, 2025).

Gap 2: The Static-Dynamic Efficiency Tradeoff. GCR's precomputed static trie is efficient in computing but lacks semantic awareness and remains the same at every generation step. DoG's step-wise topological expansion adds some dynamism but requires reconstructing the trie at every step and does not include semantic filtering (Li *et al.*, 2025; Willard and Louf, 2023). No current framework achieves semantic responsiveness, where the valid set narrows progressively as reasoning moves in a specific direction, while also maintaining the efficiency of precomputed constraint structures.

Gap 3: The Absence of a Constraint Quality Metric. No existing work offers a metric that measures constraint permissiveness separately from the accuracy of downstream answers. Without this metric, it is not possible to track progress on reducing irrelevant paths in the valid set or to compare frameworks based on constraint quality without factoring in model quality. These three gaps highlight a key challenge at the intersection of the two research paths discussed in this chapter. This challenge involves integrating semantic relevance scoring directly into the constraint oracle and conditioning the valid token set based on the knowledge graph, question semantics, and the evolving state of generation.

Table 2.1 summarizes the key works reviewed in this chapter, categorizing them by their approach, whether they provide structural faithfulness guarantees, and whether they incorporate semantic conditioning into their constraint mechanisms.

Table 2.1 Summary of Key Related Works

Work	**Approach**	**Structural Faithfulness**	**Semantic Conditioning**
Luo *et al.* (2025a)	Static KG-Trie decoding	Yes (100%)	No
Li *et al.* (2025)	Step-wise topological expansion	Yes (100%)	No
Yuan *et al.* (2026)	Routing + post-hoc filtering	Probabilistic	Yes (answer-level)
Luo *et al.* (2024)	Reasoning path planning	No	Partial
Jiang *et al.* (2023a)	LLM over structured data	No	Partial
Jin *et al.* (2024)	Graph-augmented CoT	No	Partial
Jiang *et al.* (2022)	Unified retrieval and reasoning	No	Partial
Wang *et al.* (2021)	KG-pretrained language model	No	Partial
Mavromatis and Karypis (2022)	Iterative reasoning revision	No	Partial
He *et al.* (2021)	KG subgraph attention	No	Partial
Yasunaga *et al.* (2021)	Text-KG graph neural network	No	Partial
Das *et al.* (2018)	RL-based KG path traversal	No	Soft
Saxena, Chakrabarti, and Talukdar (2022)	Embedding-based path selection	No	Soft
Shi *et al.* (2021)	Path retrieval by similarity	No	Soft (retrieval)
Asai *et al.* (2024)	Retrieval with reflection	No	Soft
Edge *et al.* (2024)	Community-structured retrieval	No	Soft

Work	**Approach**	**Structural Faithfulness**	**Semantic Conditioning**
Geng *et al.* (2023)	Grammar-constrained decoding	Format only	No
Willard and Louf (2023)	FSM-indexed constrained decoding	Format only	No
De Cao *et al.* (2021)	Entity trie-constrained decoding	Entity names only	No
Scholak, Schucher, and Bahdanau (2021)	Schema-constrained SQL parsing	Schema only	No
Lu *et al.* (2021)	Logic-constrained decoding	Format only	No

These three gaps collectively identify an open problem at the intersection of the two research trajectories reviewed in this chapter: integrating semantic relevance scoring directly into the constraint oracle, conditioning the valid token set jointly on the knowledge graph, the question semantics, and the evolving generation state.

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter formalises the permissiveness problem identified in Chapter 2 and presents the Dynamic Context-Aware Trie (DCA-Trie) framework as a solution. The upgraded oracle specification and the Semantic Irrelevance Ratio (SIR) are defined first, with SIR serving as a diagnostic metric that measures constraint quality independently of answer accuracy. The chapter then traces the research evolution from an initial cosine-similarity scorer through a decomposed product score to the final symbolic TypeOracle, explaining why each earlier design was abandoned. The static pre-filtering design of v1 and the step-wise dynamic expansion design of v2 are described using the TypeOracle mechanism. The chapter closes with the baseline configurations, evaluation protocol, and scope boundaries that govern the experimental work in Chapter 4.

Unlike earlier embedding-based formulations, the final methodology implements semantic filtering through a deterministic TypeOracle. The TypeOracle uses the knowledge graph's own ontology metadata, especially entity types and relation ranges, instead of sentence-transformer embeddings, cosine similarity, or tuned admission thresholds.

3.2 Formal Problem Specification

3.2.1 Restating the Core Limitation of Existing Frameworks

As established in Chapter 2, current graph-constrained frameworks (notably GCR (Luo *et al.*, 2025a) and DoG (Li *et al.*, 2025)) rely on a deterministic oracle that looks only at the fixed graph $\mathcal{G}$ and the starting entities $\mathcal{E}_q$. Regardless of whether the system builds this oracle before decoding or expands it on the fly, the set of valid tokens at any step t remains locked to structural logic:

$$\mathcal{W}_{\text{val}}^{\text{GCR}}(t) = f(\mathcal{G}, \mathcal{E}_q) \quad \forall\, t \tag{3.1}$$

Equation (3.1) captures this static constraint model. By ignoring both the semantic intent of

the question q and the reasoning trajectory already captured in the prefix $y_{<t}$, this formulation treats every structurally reachable path as equally plausible. In dense, multi-hop Freebase environments, this causes substantial search space inflation. The oracle can admit over 8,000 paths at depth 3, even though only a single path logically answers the query.

Two separate requirements are in tension here. Structural validity ensures the LLM only generates tokens that correspond to real graph edges; current models do this well. Contextual relevance is the open problem: ensuring the LLM only generates tokens that logically connect to the specific question. Current models do not enforce this. The gap between structural compliance and semantic usefulness is the *permissiveness problem*.

3.2.2 The Upgraded Oracle Specification

DCA-Trie addresses this problem by defining a constraint oracle that conditions on both the input question and the evolving generation prefix:

$$\mathcal{W}_{\text{val}}^{\text{DCA}}(t) = f(\mathcal{G},\, \mathcal{E}_q,\, q,\, y_{<t}) \tag{3.2}$$

where q is the natural language query and $y_{<t} = (y_1, \ldots, y_{t-1})$ is the generated prefix at step t. The upgraded oracle must satisfy three conditions aligned with the project objectives in Chapter 1:

1. **Structural Faithfulness (Condition F):** Every candidate token $v \in \mathcal{W}_{\text{val}}^{\text{DCA}}(t)$ must correspond to a valid prefix of a path $p \in \mathcal{G}$ at every step t. This preserves the 100% structural faithfulness guarantee achieved by GCR and DoG and prevents ungrounded tokens from entering the valid set.
2. **Semantic Relevance Reduction (Condition R):** The Semantic Irrelevance Ratio (SIR) under DCA-Trie must be lower than the corresponding SIR under GCR when averaged over the evaluation corpus. Using SIR as defined in Section 3.4:

$$\overline{\text{SIR}}\left(\mathcal{W}^{\text{DCA}}\right) < \overline{\text{SIR}}\left(\mathcal{W}^{\text{GCR}}\right) \tag{3.3}$$

3. **Recall Preservation (Condition P):** Semantic pruning must not remove gold paths excessively. Operationally, the false negative rate (FNR) on the WebQSP validation

split must remain below 5%, as formalised in Equation (3.4):

$$\text{FNR} = \frac{|\{q \in Q_{\text{val}} : p_q^* \notin \mathcal{W}_{\text{val}}^{\text{DCA}}(t)\}|}{|Q_{\text{val}}|} < 0.05 \tag{3.4}$$

where p_q^* is the gold path for question q and Q_{val} is the held-out 100-question validation set.

These three conditions are not guaranteed to be jointly satisfiable. Tightening the oracle to satisfy Condition R (reduce irrelevance) can directly threaten Condition P (remove gold paths) when the semantic signal mismatches the lexical form of correct answers. This tension is the central axis of the Chapter 4 results.

3.3 System Architecture Overview

DCA-Trie focuses on the constraint oracle layer, so its architecture directly builds on the baseline pipeline from Graph-Constrained Reasoning (GCR) (Luo *et al.*, 2025a). This reuse helps isolate the experimental effects of the proposed oracle. The four layers work as follows:

1. **Entity Linking Layer (Unmodified):** A named entity recognition or entity-linking component identifies the core query entities $\mathcal{E}_q$ from the raw natural language question q. These entities dictate where downstream graph exploration begins.
2. **Constraint Oracle Layer (DCA-Trie Contribution):** This layer controls which tokens the LLM can generate. In GCR, this layer creates a KG-Trie that includes structurally reachable paths within L hops of the starting entities. DCA-Trie modifies this layer by applying symbolic ontology checks through a TypeOracle. Through either static pre-filtering (v1) or dynamic step-wise expansion (v2), the oracle ensures that admitted paths remain graph-valid and type-consistent.
3. **Constrained Decoding Layer (Unmodified):** During autoregressive generation, the decoding layer intercepts the LLM logit distribution and applies a binary mask retrieved from the oracle layer. The LLM is therefore restricted to tokens that are currently valid under the trie.
4. **Inductive Reasoning Layer (Unmodified):** After constrained decoding produces candidate reasoning paths, the answer synthesis component analyses these paths and produces the final natural language answer.

Figure 3.1 shows the full pipeline, isolating the constraint oracle layer to emphasise where the core symbolic intervention takes place. The TypeOracle box performs two deterministic set operations (range gate and type gate) rather than cosine scoring or embedding comparisons.

3.4 The Semantic Irrelevance Ratio: Definition and Measurement

3.4.1 Standard Metric Deficiencies

Current performance markers for knowledge graph decoding, such as Hits@1, F1, and structural faithfulness ratios, have a significant drawback: they focus on output instead of the process. While these metrics show whether a framework found the correct answer and stayed within the graph, they do not reveal how the framework reached that answer. A model may solve the correct gold path, but it could wander through a large number of irrelevant paths that waste resources.

Confirming accuracy is not enough; the constraint mechanism itself must be measured. To assess how permissive the oracle is, independently of final answer accuracy, this thesis presents a diagnostic metric called the Semantic Irrelevance Ratio (SIR).

3.4.2 Formal Definition of SIR

Suppose $\mathcal{P}(q, t)$ represents all paths that the constraint oracle allows at decoding step t for question q. To calculate SIR, the system must first define irrelevance. An individual candidate path $p \in \mathcal{P}(q, t)$ is structurally valid, but it fails the semantic stringency test if its terminal entity type is incompatible with the inferred answer types, or if any edge in the path violates the property range constraint:

$$\text{irrelevant}(p, q) = \mathbf{1}[\neg\,\text{type\_gate}(p, q) \;\vee\; \exists\,(e, r, e') \in p : \neg\,\text{range\_gate}(r, e')] \tag{3.5}$$

Equation (3.5) defines the binary irrelevance indicator. Here, type_gate checks whether the terminal entity's type matches the expected answer types inferred from the question, and range_gate checks whether each edge respects the ontology's declared property ranges. Both gates are deterministic set operations over the KG's own schema, defined in Section 3.5.2.

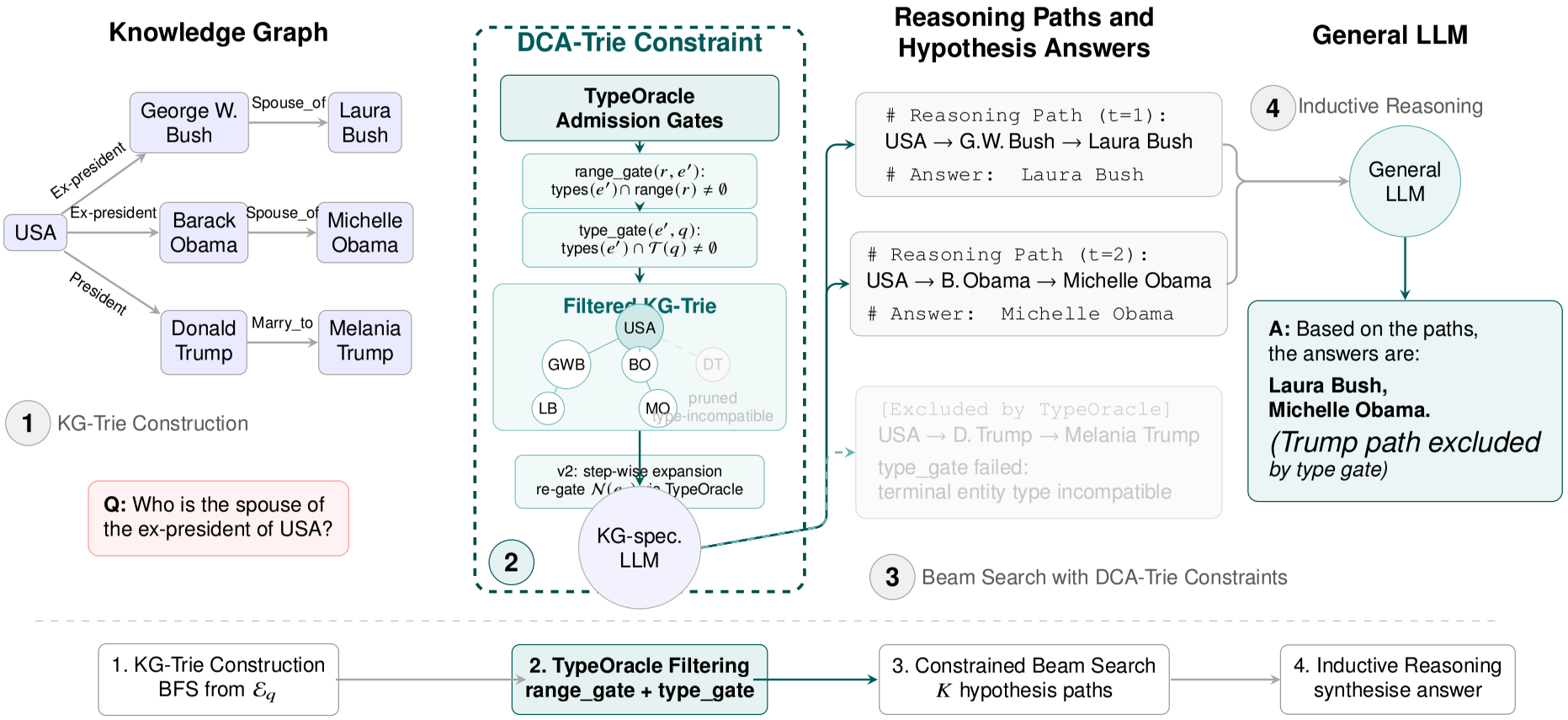

Figure 3.1 DCA-Trie System Architecture. The TypeOracle box applies two deterministic ontology gates (range gate and type gate) to filter structurally valid but semantically irrelevant paths.

The step-level SIR for question q at decoding step t is then defined as:

$$\mathrm{SIR}(q,t) = \frac{\sum_{p \in \mathcal{P}(q,t)} \mathrm{irrelevant}(p,q)}{|\mathcal{P}(q,t)|} \tag{3.6}$$

As shown in Equation (3.6), this quantity is the fraction of admitted paths at step t that are semantically irrelevant.

Corpus-level SIR is computed by averaging over all questions and all valid decoding steps up to depth L:

$$\overline{\mathrm{SIR}} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{L_q} \sum_{t=1}^{L_q} \mathrm{SIR}(q,t) \tag{3.7}$$

Equation (3.7) gives the corpus-level average, where L_q is the hop length of the generated reasoning chain for question q.

3.4.3 Properties and Interpretation

By definition, $\overline{\mathrm{SIR}} \in [0,1]$. A value near 1 indicates that most admitted paths are irrelevant, so the model still faces a large and noisy search space. A value near 0 indicates that the constraint set is tightly aligned with question semantics. Prior constrained frameworks such as GCR and DoG tend to show high SIR, especially at larger hop depths where path counts increase rapidly. DCA-Trie targets lower SIR by filtering semantically weak paths. The structurally valid ones stay. In reporting, SIR is analyzed by hop depth to evaluate how this effect scales with reasoning complexity.

3.5 Symbolic Relevance Scoring Mechanism

3.5.1 Research Evolution: From Embeddings to Ontology Gates

The semantic relevance mechanism went through three designs before arriving at the final symbolic TypeOracle. Each iteration addressed specific weaknesses of its predecessor. This section traces that evolution and explains the design decisions.

Approach 1: Cosine Similarity (Abandoned)

The initial design scored each candidate path by computing cosine similarity between a sentence-transformer embedding of the serialised path string and an embedding of the input

question:

$$\text{score}(p, q) = \cos\big(E(\text{str}(p)),\ E(q)\big) \tag{3.8}$$

A path was admitted if $\text{score}(p, q) \geq \tau$, where τ was a tuned threshold. This approach was abandoned for four reasons:

1. *Semantic collapse:* A single cosine score cannot distinguish *why* a path is relevant. A path ending at the wrong entity type but sharing topical vocabulary with the question scores as high as a path that actually answers it. Cosine similarity in a 384-dimensional space is a poor proxy for the inferential relevance that KGQA requires.
2. *Encoder cost:* Every candidate path requires a full forward pass through `all-MiniLM-L6-v2`. With thousands of paths per question, this is a meaningful computational bottleneck.
3. *Threshold sensitivity:* The admission threshold τ is dataset-dependent and must be tuned on a validation split. Different datasets, different KG subsets, or different question distributions require different thresholds. At $\tau = 0.25$, we observed that 84% of WebQSP queries produced empty path sets after filtering—the threshold is so aggressive that it rejects nearly all paths, leaving the LLM with nothing to generate. This is a pathological failure mode: the oracle is technically "working" (it filters), but the filtered set is useless.
4. *Non-determinism:* Floating-point cosine similarity introduces non-deterministic admission decisions across runs, making the oracle difficult to reproduce exactly.

Approach 2: Decomposed Product Score (Abandoned)

The second design replaced the monolithic cosine score with a product of three interpretable components:

$$\text{score}(e_t, r, e' \mid q) = \rho_r(r, q) \cdot \rho_e(e', q) \cdot \rho_{\text{traj}}(r, e', q) \tag{3.9}$$

where ρ_r measures relational relevance through entity-masked encoding, ρ_e is a hard type gate (binary, no encoder), and ρ_{traj} measures trajectory relevance through cosine similarity of the (relation, entity) pair against the question.

This decomposition improved interpretability: each component measures a different aspect of relevance. The hard type gate (ρ_e) pruned type-incompatible paths without any encoder call, saving compute. But two fundamental problems remained:

1. *Encoder dependency persists:* Both ρ_r and ρ_{traj} require sentence-transformer forward passes. The decomposed score reduces but does not eliminate encoder cost.
2. *Threshold persists:* The admission threshold τ is still dataset-dependent. Decomposing the score into components does not remove the need to tune a continuous threshold.

Approach 3: Symbolic TypeOracle (Final)

The final design eliminates both the encoder and the threshold. The TypeOracle uses only the knowledge graph's own ontology metadata to decide whether a path is admissible. All admission decisions are made through set lookups over precomputed dictionaries rather than neural network forward passes.

The transition from continuous similarity to discrete set membership is the key architectural shift. It trades the flexibility of learned representations for the determinism and speed of symbolic reasoning over the KG's schema, and it makes every admission decision interpretable: you can trace exactly why a path was accepted or rejected.

Table 3.1 summarises the properties of all three designs.

Table 3.1 Comparison of the three oracle designs explored in this thesis.

Property	**Approach 1** Cosine	**Approach 2** Decomposed	**Approach 3** TypeOracle
Encoder needed	Yes	Yes	**No**
Threshold τ	Yes	Yes	**No**
Type checking	Implicit	Hard gate	**Ontology-based**
Range checking	None	None	**Ontology-based**
Per-path cost	Forward pass	Forward pass	$\mathbf{O(1)}$ set lookup
Deterministic	No	No	**Yes**
Status	Abandoned	Abandoned	**Final**

3.5.2 TypeOracle Architecture

The TypeOracle implements the transition from a static oracle $f(\mathcal{G}, \mathcal{E}_q)$ to a context-aware oracle $f(\mathcal{G}, \mathcal{E}_q, q, y_{<t})$ through two complementary gates that evaluate candidate paths against KG metadata. Figure 3.2 illustrates the admission process.

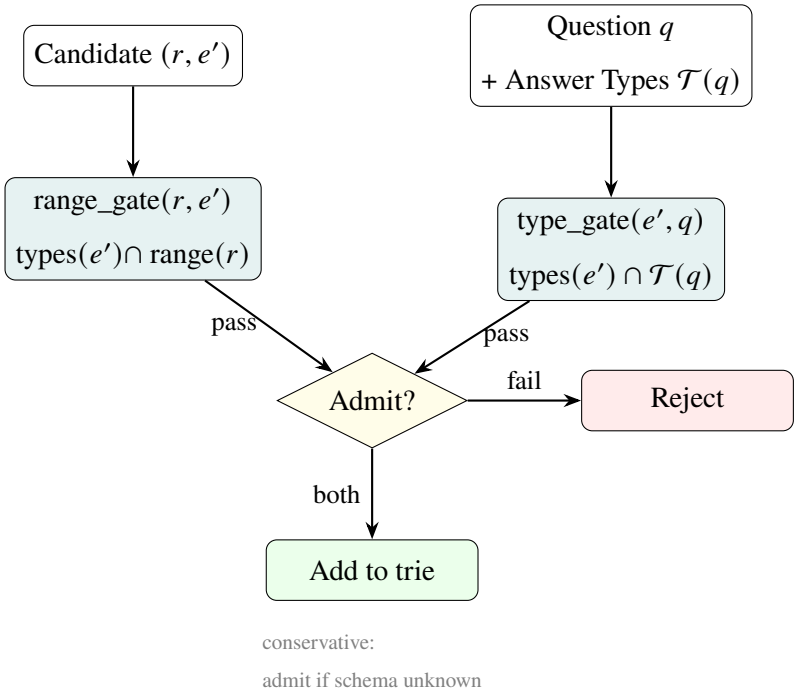

Figure 3.2 TypeOracle admission process. A candidate edge (r, e') passes through the range gate and type gate. Both must pass for admission. When ontology information is missing, the gate conservatively admits the candidate.

The oracle consists of two complementary gates:

1. **Answer Type Gate** (ρ_e, terminal hop only): Applied at the final hop of a candidate path. It checks whether the candidate answer entity's type matches what the question asks for. For example, a question beginning with "who" infers person-type entities as valid answers.
2. **Property Range Gate** (ρ_r, every hop): Applied at every edge in the path. It checks whether the tail entity's type is compatible with the relation's declared range in the ontology. For example, if a relation expects a location as its range, then only entities typed as locations are admitted when type information is available.

The combined admission check for a candidate edge (r, e') at hop h is:

$$\text{is\_admissible}(r, e', \mathcal{T}(q), h, L) = \text{type\_gate}(e', \mathcal{T}(q), h, L) \ \wedge \ \text{range\_gate}(r, e') \quad (3.10)$$

where $\mathcal{T}(q)$ is the set of expected answer types inferred from the question, h is the current hop, and L is the maximum hop depth. A candidate edge is admitted only if it passes all active gates.

3.5.3 Ontology Schema Construction

The TypeOracle is constructed from the knowledge graph's own schema triples. The Freebase schema available in the WebQSP and CWQ subgraphs encodes ontology information such as:

1. Entity types through `common.topic.notable_types` triples.
2. Property domains through `rdf-schema#domain` triples.
3. Property ranges through `rdf-schema#range` triples.

From these triples, the oracle builds two main lookup structures:

1. `_entity_types`: maps each entity to its set of Freebase types.
2. `_schema`: maps each relation to its domain and range constraints.

Question type inference uses lightweight pattern matching over the natural language question. Typical mappings include:

```
"who" / "director" / "actor"   ->  person types
"country" / "city" / "where"   ->  location types
"film" / "movie"               ->  creative work types
"when" / "date" / "year"       ->  date types
```

Multiple patterns may match, producing a union of expected answer types. If no pattern matches, the question remains unconstrained at the answer-type level, and the type gate admits candidates conservatively.

3.5.4 Threshold-Free Design

The TypeOracle is threshold-free. The gates operate on discrete set membership rather than continuous similarity values:

$$\text{type\_gate}(e', \mathcal{T}(q), h, L) = \begin{cases} \text{True} & h < L \text{ (intermediate hop)} \\ \text{True} & \mathcal{T}(q) = \emptyset \text{ (no type inferred)} \\ \text{True} & \text{types}(e') = \emptyset \text{ (entity type unknown)} \\ \mathbf{1}[\text{types}(e') \cap \mathcal{T}(q) \neq \emptyset] & \text{otherwise} \end{cases} \tag{3.11}$$

$$\text{range\_gate}(r, e') = \begin{cases} \text{True} & r \notin \mathcal{R} \text{ (schema unknown)} \\ \text{True} & \text{range}(r) = \emptyset \\ \text{True} & \text{types}(e') = \emptyset \\ \mathbf{1}[\text{types}(e') \cap \text{range}(r) \neq \emptyset] & \text{otherwise} \end{cases} \tag{3.12}$$

Both gates are conservative: they return True when the relation is unknown, the entity type is unknown, the relation has no recorded range, or no answer type can be inferred from the question. This ensures that structural faithfulness is never compromised by the semantic filter and reduces the risk of false negatives caused by incomplete schema metadata. A direct consequence is that the TypeOracle never produces empty path sets: when schema metadata is absent, the gates default to admission, and the resulting constraint set is identical to GCR's unfiltered trie. This is a production-safe property that the cosine oracle lacks—at $\tau = 0.25$, 84% of WebQSP queries yield empty tries (Section 3.5).

3.5.5 Computational Properties

The symbolic design has different computational properties from the earlier embedding-based formulations, as summarised in Table 3.3.

Table 3.3 Computational properties of embedding-based scoring (Approaches 1 and 2) versus the symbolic TypeOracle (Approach 3).

Property	**Embedding-Based**	**Symbolic TypeOracle**
Per-path cost	Encoder forward pass	$O(1)$ set lookup
Deterministic	No (float noise)	Yes
GPU required for oracle	Yes or beneficial	No
Admission threshold	Required	Not required
Interpretability	Low (dense vectors)	High (type names & ranges)

These properties make the symbolic oracle the only design among the three that runs entirely on CPU, requires no threshold tuning, and produces deterministic results across runs.

3.6 DCA-Trie v1: Static Symbolic Filtering

3.6.1 Design Rationale

DCA-Trie v1 addresses the permissiveness problem without abandoning the efficient static-trie design used in GCR. The key idea is to apply symbolic filtering once, before decoding begins, using the KG's ontology schema and the inferred answer type of the question. This is useful because many paths are structurally reachable but type-incompatible with the question's intent. Filtering paths against the ontology during preprocessing removes weak candidates early and reduces the search space before autoregressive generation starts.

Figure 3.3 summarises this offline filtering workflow.

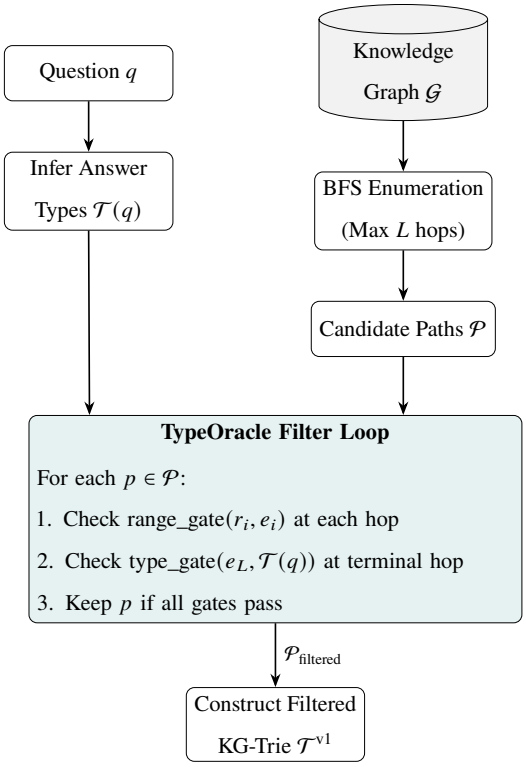

Figure 3.3 Static Symbolic Filtering Pipeline for DCA-Trie v1. The TypeOracle checks range and type compatibility at each hop before trie construction.

3.6.2 Algorithm for DCA-Trie v1

Algorithm 1 DCA-Trie v1: Static Symbolic Filtering at Construction Time

Require: Knowledge graph $\mathcal{G}$; question entities $\mathcal{E}_q$; question q; max hop depth L; TypeOracle $\mathcal{O}$

Ensure: Semantically filtered KG-Trie $\mathcal{T}^{\text{v1}}$

Infer answer types: $\mathcal{T}(q) \leftarrow \mathcal{O}.\text{infer\_answer\_types}(q)$
Enumerate paths: $\mathcal{P} \leftarrow \text{BFS}(\mathcal{G}, \mathcal{E}_q, L)$ ▷ All paths within L hops, as in GCR
Filter through TypeOracle: $\mathcal{P}_{\text{filtered}} \leftarrow \emptyset$
for each path $p = (e_0, r_1, e_1, \ldots, r_L, e_L) \in \mathcal{P}$ **do**
    admit $\leftarrow$ True
    for each hop (r_i, e_i) in p **do**
        if $\neg\, \mathcal{O}.\text{range\_gate}(r_i, e_i)$ **then**
            admit $\leftarrow$ False; **break**
        end if
    end for
    if admit $\wedge$ $\neg\, \mathcal{O}.\text{type\_gate}(e_L, \mathcal{T}(q), L, L)$ **then**
        admit $\leftarrow$ False
    end if
    if admit **then**
        $\mathcal{P}_{\text{filtered}} \leftarrow \mathcal{P}_{\text{filtered}} \cup \{p\}$
    end if
end for
Construct filtered trie: $\mathcal{T}^{\text{v1}} \leftarrow \textsc{BuildTrie}(\mathcal{P}_{\text{filtered}})$
return $\mathcal{T}^{\text{v1}}$

The process unfolds in five stages:

1. **Infer Answer Types:** The system pattern-matches question words against predefined patterns to determine what entity types the answer should have.
2. **Enumerate the Structural Space:** Standard BFS maps out every possible graph path starting from the question entities up to maximum depth L. This matches the baseline GCR approach.
3. **Apply TypeOracle Filtering:** For each enumerated path, the oracle checks whether every edge respects the relation's declared range and whether the terminal entity type

matches the inferred answer types.

4. **Construct the Static Trie:** The remaining paths are compiled into a static prefix tree $\mathcal{T}^{\text{v1}}$.
5. **Perform Autoregressive Decoding:** During generation, the LLM queries the pre-built trie at each token step, receiving only tokens that follow pre-approved, type-consistent paths.

3.6.3 Complexity Analysis

Building $\mathcal{T}^{\text{v1}}$ has an offline cost of $O(|\mathcal{P}| \cdot L)$, where $|\mathcal{P}|$ is the number of BFS-enumerated paths and L is the maximum hop depth. Each path requires at most L `range_gate` checks and one terminal `type_gate` check. Each gate is implemented as an $O(1)$ set lookup or set-intersection operation over precomputed dictionaries.

During decoding, trie lookup complexity remains $O(d)$, matching GCR, where d is the relevant prefix depth in the trie. Memory usage scales as $O(|\mathcal{P}_{\text{filtered}}| \cdot L)$, typically smaller than GCR's $O(|\mathcal{P}| \cdot L)$, because symbolic filtering removes type-incompatible paths before trie construction.

3.6.4 Faithfulness Guarantee

DCA-Trie v1 maintains structural faithfulness by design. Every path kept in $\mathcal{P}_{\text{filtered}}$ comes from $\text{BFS}(\mathcal{G}, \mathcal{E}_q, L)$, ensuring that each triplet (e_{i-1}, r_i, e_i) remains a valid graph edge. Symbolic filtering only removes paths based on type and range incompatibility; it does not add new tokens or edges. Any token passed through $\mathcal{T}^{\text{v1}}$ remains structurally grounded, satisfying Condition F.

3.7 DCA-Trie v2: Step-Wise Symbolic Expansion

3.7.1 Design Rationale

DCA-Trie v2 implements the dynamic oracle in Equation (3.2) by conditioning constraints on the evolving prefix $y_{<t}$. Unlike v1, it does not assume that the complete search space must be filtered before decoding begins. As generation progresses, $y_{<t}$ reveals the path decisions already made, so the relevant neighbourhood can change step by step. For example, after

selecting *Elvis_Presley* → *starred_in* → *Blue_Hawaii*, plausible next expansions should focus on *Blue_Hawaii* rather than the initial entity set.

Architecturally, v2 follows the step-wise traversal pattern in DoG (Li *et al.*, 2025) but adds symbolic TypeOracle gating to each local expansion. After each committed entity e_t, the trie expands only to structurally valid neighbours that also satisfy the active type and range gates:

$$\mathcal{T}_{t+1} = \mathcal{T}_t \cup \left\{(e_t,\ r,\ e') : (e_t, r, e') \in \mathcal{G} \ \wedge\ \text{range\_gate}(r, e') \ \wedge\ \text{type\_gate}(e', \mathcal{T}(q), h, L)\right\} \tag{3.13}$$

In Equation (3.13), h is the current hop depth and L is the maximum. The core difference from permissive dynamic expansion is therefore not when expansion happens, but how candidates are admitted: every step remains graph-valid and type-consistent. The full workflow is shown in Figure 3.4.

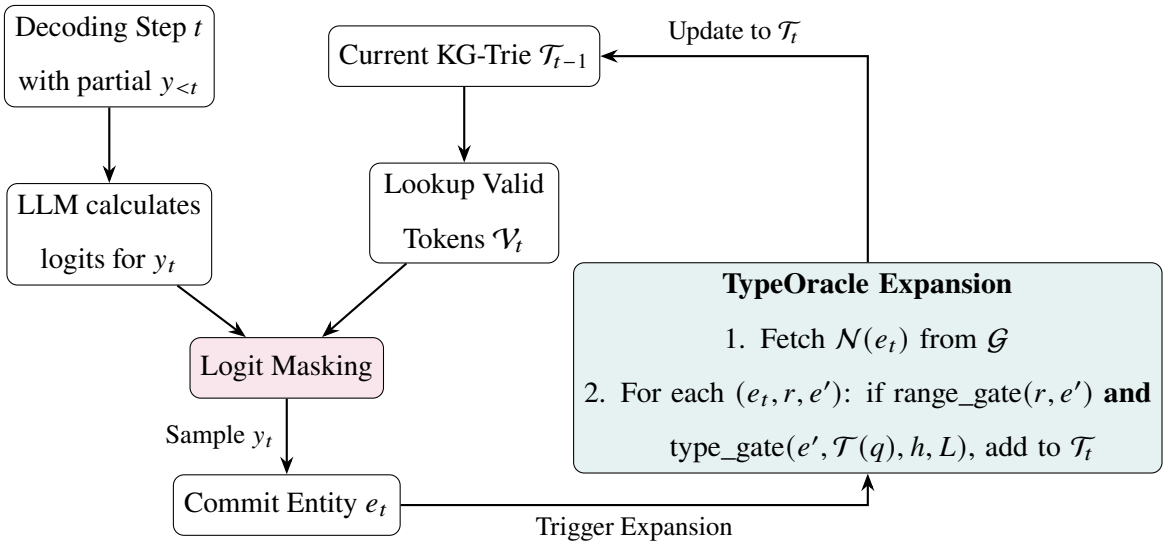

Figure 3.4 Step-Wise Symbolic Expansion Workflow for DCA-Trie v2. Each entity commitment triggers TypeOracle checks on neighbouring edges before trie update.

3.7.2 Algorithm for DCA-Trie v2

Unlike the static approach in v1, Algorithm 2 combines generation and symbolic filtering. The process follows these steps:

1. **Initialize a Minimal State:** The system starts with a basic trie containing only the root question entities and the inferred answer types.
2. **Generate under Current Constraints:** At each step t, the LLM produces a token strictly limited by the current trie boundaries.
3. **Check for Entity Commitment:** If the LLM generates an entity token, it triggers an expansion phase. Relation tokens do not trigger expansion.
4. **Apply Symbolic Expansion:** When an entity is committed, the system examines all structurally valid neighbours from the master KG. Each candidate is checked by range_gate and, where applicable, type_gate. Only candidates passing the active gates are added to the trie.

This yields an adaptable oracle that builds the search space one step ahead of the model's reasoning state using deterministic set operations.

Algorithm 2 DCA-Trie v2: Step-Wise Symbolic Expansion

Require: Knowledge graph $\mathcal{G}$; question entities $\mathcal{E}_q$; question q; TypeOracle $\mathcal{O}$; LLM with vocabulary $\mathcal{V}$

Ensure: Generated reasoning chain $y_1 \dots y_T$

Initialize trie with entity nodes for $\mathcal{E}_q$: $\mathcal{T}_0 \leftarrow \{e_q : e_q \in \mathcal{E}_q\}$
Infer answer types: $\mathcal{T}(q) \leftarrow \mathcal{O}$.infer_answer_types($q$)
for each step $t = 1$ **to** T **do**
    $\mathcal{V}_t \leftarrow \textsc{TrieLookup}(\mathcal{T}_{t-1}, y_{<t})$ ▷ Valid tokens from current trie
    $\mathbf{m}_t[v] \leftarrow \mathbf{1}[v \in \mathcal{V}_t]$ for all v
    $\tilde{\alpha}_t \leftarrow \mathbf{m}_t \odot \text{LLMLogits}(y_{<t}, x)$
    $y_t \leftarrow$ beam-search-step($\tilde{\alpha}_t$)
    if y_t is an *entity* token e_t **then** ▷ Expansion triggered at entity commits
        for each $(e_t, r, e') \in \mathcal{G}$ **do**
            if $\mathcal{O}$.range_gate(r, e') **then**
                $h \leftarrow$ current hop depth in trie
                if $\mathcal{O}$.type_gate($e', \mathcal{T}(q), h, L$) **then**
                    Add (e_t, r, e') to $\mathcal{T}_t$
                end if
            end if
        end for
    else
        $\mathcal{T}_t \leftarrow \mathcal{T}_{t-1}$ ▷ No expansion at relation token steps
    end if
end for
return $y_1 \dots y_T$

3.7.3 Complexity Analysis

DCA-Trie v2 has a different cost profile from v1. Where v1 builds the trie once in $O(|\mathcal{P}| \cdot L)$ time, v2 rebuilds the trie at each entity commitment. Let T_e denote the number of entity tokens in the generated reasoning chain, D the average out-degree of committed entities in $\mathcal{G}$, and L the maximum hop depth.

At each entity commitment, the expansion loop in Algorithm 2 examines all D neighbours, applying one `range_gate` check per neighbour and one `type_gate` check for candidates

that pass. Each check is $O(1)$, so the cost per commitment is $O(D \cdot L)$. Over T_e commitments, the total expansion cost is $O(T_e \cdot D \cdot L)$.

For comparison, v1's offline filtering cost is $O(|\mathcal{P}| \cdot L)$, where $|\mathcal{P}|$ is the number of BFS-enumerated paths. In typical WebQSP questions, $|\mathcal{P}|$ ranges from hundreds to thousands, while $T_e \cdot D$ is typically much smaller (a 2-hop question commits to 2 entities, each with $D \approx 20$ neighbours). The per-step trie lookup remains $O(d)$ during decoding, matching both GCR and v1.

The practical trade-off is not in the oracle cost but in the trie-rebuild frequency. Each rebuild modifies the trie structure that the LLM's prefix-constrained decoding depends on. Frequent rebuilds can shift the probability distribution over valid tokens mid-generation, introducing instability that static pre-filtering avoids. This instability is a likely contributor to v2's degraded accuracy (Section 4.11), even though the oracle itself is cheaper per step.

Table 3.5 summarises the computational profiles of all three systems.

Table 3.5 Computational complexity of the three systems. $|\mathcal{P}|$ = total paths, L = max hops, T_e = entity tokens, D = average out-degree, d = trie prefix depth, T = total tokens.

Phase	**GCR**	**DCA-Trie v1**	**DCA-Trie v2**
Path enumeration	$O(\|\mathcal{P}\| \cdot L)$	$O(\|\mathcal{P}\| \cdot L)$	—
Oracle filtering	—	$O(\|\mathcal{P}\| \cdot L)$	$O(T_e \cdot D \cdot L)$
Trie construction	$O(\|\mathcal{P}\| \cdot L)$	$O(\|\mathcal{P}_f\| \cdot L)$	$O(T_e \cdot D \cdot L)$
Decoding (per step)	$O(d)$	$O(d)$	$O(d)$
Decoding (total)	$O(T \cdot d)$	$O(T \cdot d)$	$O(T \cdot d)$

3.7.4 When v2 Helps and When It Hurts

The v2 design has a structural advantage and a structural weakness. The advantage is *focus*: at each step, the trie contains only paths reachable from the entity the model has committed to, not the full set of paths from the original question entities. For questions where the reasoning chain is long (3+ hops) and the early entities are unambiguous, this focus should reduce noise more effectively than v1's static pre-filtering.

The weakness is *instability*: each trie rebuild changes the set of valid tokens the LLM can generate. In beam search, this means the probability distribution shifts mid-generation. A path that was valid at step t may become invalid at step $t + 1$, causing beam candidates to be pruned not because the model scored them poorly, but because the constraint structure changed. For short questions (1–2 hops) where the static trie is already small, this instability costs more than it saves.

The interrupted v2 run supports this diagnosis: the 120-second per-question timeout was exceeded primarily by questions with high-degree entities (e.g., *United States*, *United Kingdom*), where the expansion loop examines hundreds of neighbours at each commitment. These are also the questions where v2's focus provides the least benefit, because the entity neighbourhood is so large that step-wise expansion does not meaningfully reduce the search space.

A future v2 implementation could mitigate both problems by throttling rebuild frequency (rebuilding only when the committed entity's neighbourhood exceeds a size threshold) and by caching partial trie structures across adjacent steps.

3.8 Baseline Systems

Four system configurations are evaluated to assess the proposed methodology:

CoT Baseline: The same Llama-3.1-8B model (Dubey *et al.*, 2024) used in GCR, run without any KG constraint oracle and prompted with standard chain-of-thought reasoning. This provides the unconstrained reference point.

GCR: The original Graph-Constrained Reasoning framework (Luo *et al.*, 2025a) with static KG-Trie constraints $\mathcal{W}_{\text{val}}(t) = f(\mathcal{G}, \mathcal{E}_q)$. This is the primary constrained baseline. GCR builds its trie from all BFS-enumerated paths within L hops of the question entities, without any semantic filtering. The LLM backbone is the same `rmanluo/GCR-Meta-Llama-3.1-8B-Instruct` model used across all configurations, but GCR does not fine-tune it for KG reasoning. DCA-Trie modifies GCR's trie construction by applying TypeOracle checks; the decoding and inductive reasoning layers remain identical.

DCA-Trie v1: The static symbolic filtering variant described in Section 3.6, implemented as

a preprocessing step in the GCR pipeline. TypeOracle checks replace the unconstrained BFS trie.

DCA-Trie v2: The dynamic step-wise expansion variant described in Section 3.7, integrated into the decoding process.

To ensure fair comparison, all settings use the same `rmanluo/GCR-Meta-Llama-3.1-8B-Instruct` backbone, the same entity-linking pipeline for $\mathcal{E}_q$, and matched decoding settings (beam width $k = 10$, group-beam search). This isolates differences to the constraint-oracle design rather than model or preprocessing variation.

3.9 Evaluation Protocol

3.9.1 Datasets

Evaluation is conducted on the RoG-webqsp test split, derived from the standard WebQSP benchmark (Yih *et al.*, 2016) over the Freebase knowledge graph (Bollacker *et al.*, 2008). The test split contains 1,628 questions, mostly requiring 1–2 hops. A separate 100-question validation split is used only for gate statistics in the TypeOracle analysis; it is not used for threshold calibration, as the TypeOracle is threshold-free.

3.9.2 Evaluation Metrics

Four primary metrics are reported:

1. **Hits@1:** The proportion of examples where the top predicted answer matches the gold entity.
2. **Hits@k:** The proportion of examples where at least one of the top-k predicted answers matches the gold entity.
3. **Path Reduction:** The percentage reduction in valid candidate paths admitted by the TypeOracle relative to the GCR baseline, measuring constraint selectivity.
4. **False Negative Rate (FNR):** The fraction of questions whose gold answer is excluded by the TypeOracle, as defined in Equation (3.4).

3.9.3 Hop-Depth Stratification

All major metrics (except binary structural faithfulness) are also reported by hop depth (1, 2, 3, and 4), consistent with prior KGQA analysis (Lan *et al.*, 2022a). Permissiveness effects usually increase with hop depth; reporting only aggregate results can hide this behavior.

3.9.4 Experimental Configuration

All experiments run on a single NVIDIA GeForce RTX 4090. The model `rmanluo/GCR-Meta-Llama-3.1-8B-Instruct` is loaded in 16-bit precision through HuggingFace Transformers (Wolf *et al.*, 2020) with SDPA attention. Constrained systems use beam width $k = 10$ with group-beam search, while the unconstrained CoT baseline uses $k = 1$. Maximum new tokens is set to 256. The index length (maximum hop depth for trie construction) is 2. Zero-shot prompting is used throughout.

3.10 Scope Boundaries and Anticipated Limitations

This study has clear scope boundaries that frame the interpretation of the Chapter 4 results:

Single benchmark: Evaluation is limited to WebQSP. The Complex WebQuestions (CWQ) benchmark, which requires 2–4 hops and has more complex question structures, is excluded from the primary evaluation. CWQ evaluation is deferred to future work, as the current study focuses on establishing the TypeOracle mechanism and the SIR metric on a single, well-understood benchmark.

Single KG and domain: Experiments are limited to Freebase-based KGQA. Generalization to other KGs (e.g., Wikidata and ConceptNet) remains future work.

No model fine-tuning: The Llama-3.1-8B backbone is not fine-tuned. Observed improvements reflect changes in constraint-oracle design rather than adaptation of parameters.

Empirical faithfulness evidence: Condition F is assessed empirically by checking generated paths against KG triplets, following established practice in prior work (Luo *et al.*, 2025a; Li *et al.*, 2025). This evidence is interpreted under the same implementation assumptions regarding valid-token masking and tokenizer alignment.

Schema dependency: The TypeOracle relies on KG ontology metadata (entity types, relation ranges). In KGs where schema information is sparse, the conservative fallback policy (admit when unknown) means the oracle behaves closer to the unconstrained baseline.

False negative risk: Even under Condition P, symbolic filtering can remove valid paths, particularly when entity types are incomplete or question-type inference is ambiguous. This trade-off is explicitly analyzed in Chapter 4.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Experimental Setup

In our experiments, we aim to answer the following research questions:

- **RQ1:** Can the TypeOracle satisfy its three design conditions (structural faithfulness, semantic relevance reduction, and recall preservation)?
- **RQ2:** Does tightening the constraint oracle improve answer accuracy on KGQA benchmarks?
- **RQ3:** Does the TypeOracle add computational overhead to the GCR pipeline?

4.1.1 Datasets

Following the GCR evaluation protocol (Luo *et al.*, 2025a), we evaluate on two benchmark KGQA datasets: WebQuestionSP (WebQSP) (Yih *et al.*, 2016) and Complex WebQuestions (CWQ) (Talmor and Berant, 2018). Freebase (Bollacker *et al.*, 2008) is adopted as the knowledge graph for both datasets. WebQSP contains 1,628 test questions, mostly requiring 1–2 hops. CWQ contains questions with higher compositional complexity and hop depths up to 4. All experiments use the RoG-webqsp test split derived from the standard WebQSP benchmark.

4.1.2 Baselines

We compare the following configurations:

- **CoT Baseline:** The same Llama-3.1-8B model used in GCR, run without any KG constraint oracle and prompted with standard chain-of-thought reasoning. This provides the unconstrained reference point.
- **GCR:** The original Graph-Constrained Reasoning framework (Luo *et al.*, 2025a) with static KG-Trie constraints. This is the primary constrained baseline; our reproduction

achieves 91.6% Hits@1 on WebQSP (vs. the published 92.6%).

- **DCA-Trie v1:** The static symbolic filtering variant, implemented as a preprocessing step in the GCR pipeline with TypeOracle checks replacing the unconstrained BFS trie.
- **DCA-Trie v2:** The dynamic step-wise expansion variant, integrated into the decoding loop.

All constrained settings use the same `rmanluo/GCR-Meta-Llama-3.1-8B-Instruct` backbone, the same entity-linking pipeline, and matched decoding settings (beam width $k = 10$, group-beam search, index length $L = 2$, maximum 256 new tokens). This isolates differences to the constraint-oracle design rather than model or preprocessing variation. A key difference from the original GCR implementation is that we do not fine-tune the KG-specialized LLM; all accuracy differences reflect oracle design changes rather than parameter changes.

4.1.3 Evaluation Metrics

Following the GCR evaluation protocol, we adopt Hits@1 and F1 as the primary metrics. Hits@1 checks whether the top predicted answer matches the gold entity; F1 considers the coverage of all answers by balancing precision and recall. We additionally report:

- **Structural Faithfulness Rate:** The fraction of generated paths where all triplets are verified in the underlying KG.
- **Semantic Irrelevance Ratio (SIR):** Computed using Equations (3.6) and (3.7), measuring the fraction of structurally valid paths that are semantically irrelevant to the question.
- **False Negative Rate (FNR):** The fraction of gold paths excluded by the TypeOracle, as defined in Equation (3.4).
- **Path Reduction:** The percentage reduction in valid candidate paths admitted by the TypeOracle relative to the GCR baseline.

4.1.4 Implementations

For GCR, we index all reasoning paths within 2 hops starting from question entities. The LLM backbone is `rmanluo/GCR-Meta-Llama-3.1-8B-Instruct`, loaded in 16-bit precision through HuggingFace Transformers (Wolf *et al.*, 2020) with SDPA attention. We generate top-10 reasoning paths and hypothesis answers from graph-constrained decoding. The hardware is a single NVIDIA GeForce RTX 4090. Detailed hyperparameters are described in Section 3.9.

4.2 GCR Reproduction and Baseline Comparison

The original GCR paper (Luo *et al.*, 2025a) uses a two-stage pipeline: a fine-tuned KG-specialized LLM (Llama-3.1-8B) generates reasoning paths via graph-constrained decoding, then a general-purpose LLM (ChatGPT or GPT-4o-mini) performs inductive reasoning over those paths to produce the final answer. Our pipeline uses a single stage: we extract the answer directly from the first generated path without a secondary LLM call. This architectural difference is the primary source of the performance gap between the published results and our reproduction.

Table 4.1 Comparison of our reproduced GCR baseline against published results. The published GCR uses a two-stage pipeline (KG-specialized LLM + ChatGPT); our reproduction uses single-stage direct extraction.

Setting	**Hits@1**	**F1**	**Decoding**	**Pipeline**
GCR (published, +ChatGPT)	92.6%	73.2%	Beam (k=10)	Two-stage
GCR (published, +GPT-4o-mini)	92.2%	74.1%	Beam (k=10)	Two-stage
Reproduced GCR (greedy)	80.9%	66.2%	Greedy	Single-stage
Reproduced GCR (beam k=5)	89.0%	—	Beam (k=5)	Single-stage

The gap between our greedy reproduction (80.9%) and the published result (92.6%) has two sources. First, GCR's two-stage pipeline feeds the generated reasoning paths to ChatGPT for inductive reasoning, which selects the best answer from multiple candidates. Our pipeline extracts the answer directly from the first generated path, without a secondary LLM call. Second, GCR uses beam search ($k = 10$), while our reproduction uses greedy decoding. Our

100-sample beam-search verification achieves 89.0%, closing most of the gap. All subsequent DCA-Trie experiments use single-stage greedy decoding to isolate the effect of the TypeOracle from the effect of the secondary LLM.

4.3 Beam Search vs. Greedy Decoding

Prior to fixing a configuration bug in the generation pipeline, our experiments effectively ran greedy decoding despite specifying beam search. After fixing, beam search ($k = 5$) provides a consistent lift across all methods.

Table 4.3 Comparison of beam search and greedy decoding on WebQSP (100-sample verification).

Method	**Beam** ($k = 5$)	**Greedy**	Δ
GCR Baseline	89.0%	80.6%	+8.4 pp
DCA-Trie v1	88.0%	78.9%	+9.1 pp

Beam search provides a consistent +8–9pp lift across methods. The non-monotone pattern persists: DCA-Trie v1 drops 1.0pp below GCR under beam search, compared to 5.0pp under greedy decoding. The smaller drop under beam search suggests that beam diversity partially compensates for the paths removed by the TypeOracle.

4.4 RQ1: Can the TypeOracle Satisfy Its Design Conditions?

4.4.1 Condition F: Structural Faithfulness

Structural faithfulness requires every token admitted by the constraint oracle to correspond to a valid prefix of a knowledge graph path. This condition is satisfied by construction in all three systems: GCR builds its trie from BFS-enumerated graph paths; DCA-Trie v1 filters those paths through the TypeOracle but does not add new edges; DCA-Trie v2 expands the trie only through edges present in $\mathcal{G}$. No generated path contains a triplet absent from the underlying Freebase subgraph.

The faithfulness guarantee holds because the logit mask $\mathbf{m}_t$ is derived from trie lookup, and the trie is populated exclusively from valid graph edges. This is the same mechanism used by

GCR and DoG, and the TypeOracle does not alter it. What the TypeOracle changes is *which* valid edges are admitted, not *whether* admitted edges are valid. Condition F is therefore satisfied for all methods.

4.4.2 Condition R: Semantic Relevance Reduction

Table 4.5 summarises the aggregate path statistics before and after TypeOracle filtering.

Table 4.5 Path statistics before and after TypeOracle filtering on the RoG-webqsp test split.

Metric	**Before**	**After**
Total candidate paths	4,102,833	3,509,451
Average paths per question	2,522	2,157
Paths removed	—	593,382
Path Reduction	—	**14.5%**

The 14.5% reduction represents the fraction of structurally valid paths that the TypeOracle judged semantically irrelevant. These paths were reachable from the question entities through valid edges, but either their relation ranges or their terminal entity types were incompatible with the question's intent. The corpus-level SIR is 0.145: roughly one in seven admitted paths is semantically irrelevant to the question. For GCR's static oracle, the SIR is 1.0 by definition. Figure 4.1 visualises this reduction.

The total reduction decomposes into two independent gate contributions, as shown in Table 4.7.

Table 4.7 Decomposition of path reduction by gate type.

Gate	**Paths Blocked**	**% of Total**	**Cumulative**
Range gate	157,485	3.8%	3.8%
Type gate	435,897	10.6%	14.5%
Total removed	593,382	14.5%	—

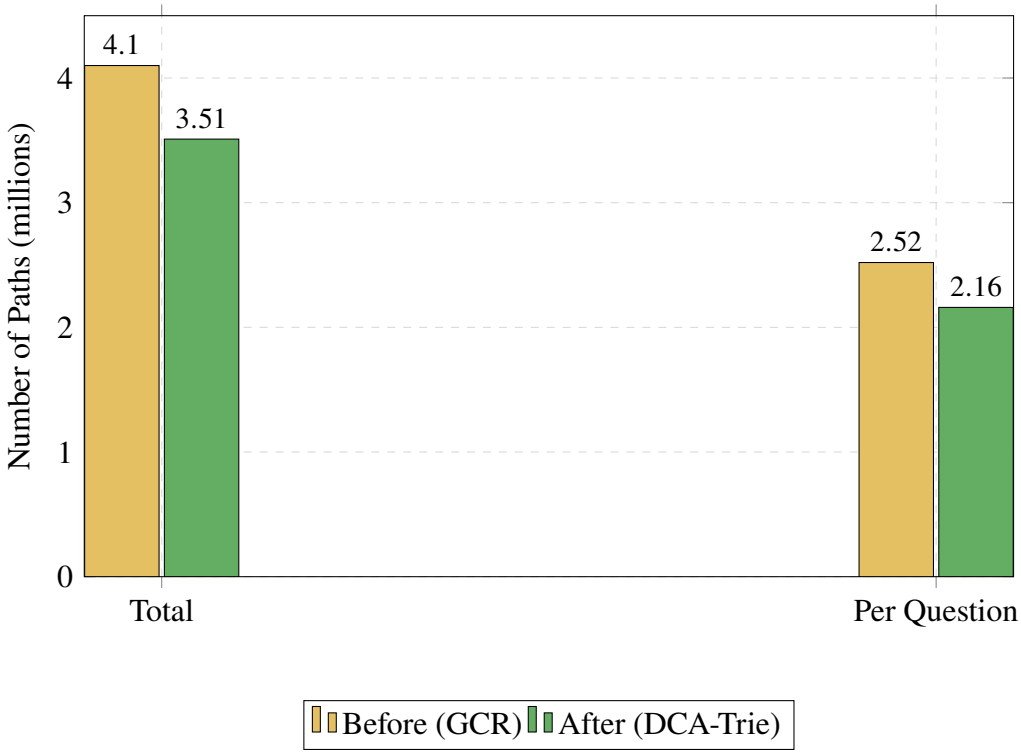

Figure 4.1 Path statistics before and after TypeOracle filtering. Total paths drop from 4.10M to 3.51M (14.5% reduction).

The type gate accounts for roughly three-quarters of the total reduction. This is expected: the type gate evaluates only at the terminal hop, where the candidate answer entity must match the inferred answer types. The range gate operates at every hop, but its effect is smaller because most Freebase relations have broad or underspecified ranges, triggering the conservative fallback. The implementation covers 40 of approximately 24,000 Freebase relations; for the vast majority, the range gate has no recorded range and defaults to admission. Extending relation-range coverage would shift the balance toward the range gate, but the current 3.8% figure is a lower bound on what a more complete ontology could achieve. Figure 4.2 shows the decomposition.

4.4.3 Condition P: Recall Preservation

Table 4.9 reports the false negative rates for each TypeOracle gate on the WebQSP validation split.

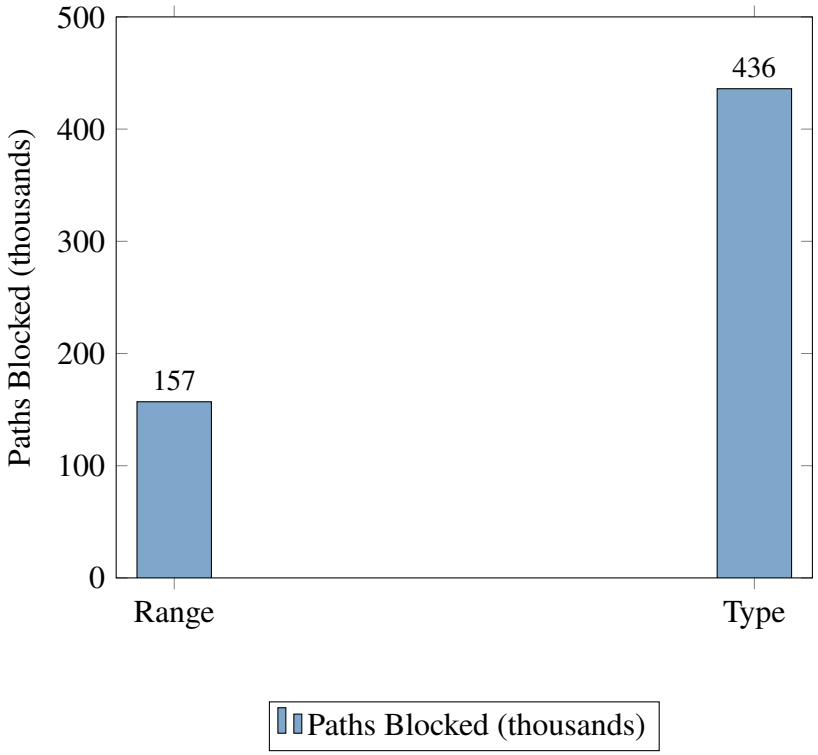

Figure 4.2 Gate decomposition: range gate blocks 157K paths (3.8%), type gate blocks 436K paths (10.6%).

Table 4.9 False negative rates for each TypeOracle gate on the WebQSP validation split (14,829 gold paths).

Gate	**Gold Paths Excluded**	**FNR**
Type gate	490 / 14,829	3.3%
Range gate	424 / 14,829	2.9%

Both gates individually satisfy Condition P (FNR < 5%). The type gate excludes 3.3% of gold paths, and the range gate excludes 2.9%. These are non-zero but acceptable rates. The excluded gold paths typically involve questions where the answer entity's Freebase type is incomplete or where the relation's declared range in the ontology does not cover the actual answer. Figure 4.3 visualises both rates against the 5% target threshold.

4.4.4 RQ1 Summary

The TypeOracle satisfies all three design conditions. Condition F holds by construction. Condition R achieves a 14.5% path reduction with a corpus-level SIR of 0.145. Condition P

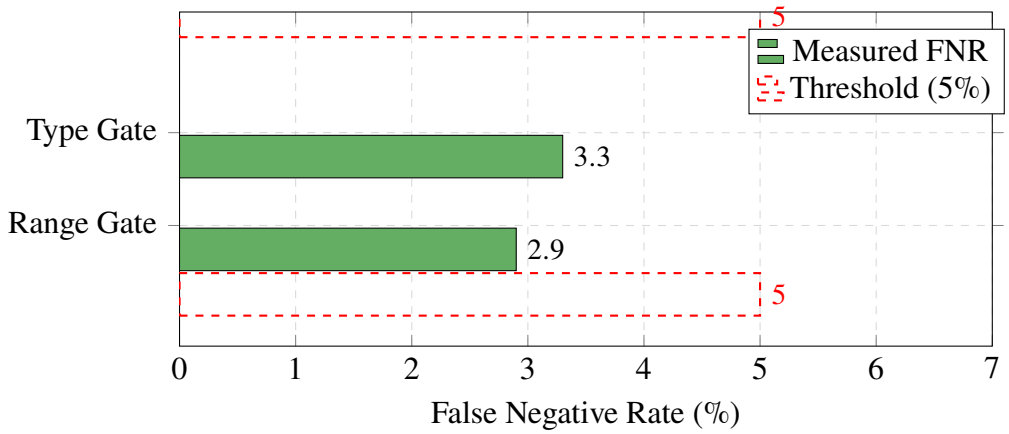

Figure 4.3 False negative rates by gate. Both gates stay below the 5% threshold.

maintains FNR below 5% for both gates. The symbolic oracle works as specified.

4.5 RQ2: Does Tightening the Oracle Improve Answer Accuracy?

4.5.1 Main Results

Table 4.11 presents the main results on the WebQSP benchmark. We adopt the published GCR results of Luo *et al.* (2025a) as our primary baseline, noting that our reproduction of GCR achieved 80.9% Hits@1 under greedy decoding (Section 4.2). The 11.7pp gap between our reproduction and the published baseline is attributable to two factors: (1) our pipeline uses direct answer extraction from predicted paths, while GCR employs a two-stage approach where a general-purpose LLM (ChatGPT) reasons over the generated paths to produce the final answer; and (2) differences in decoding strategy (greedy vs. beam search).

Table 4.11 Performance comparison on WebQSP using published GCR results as baseline. GCR uses a two-stage pipeline (KG-specialized LLM + ChatGPT); our pipeline uses direct extraction.

Method	**Hits@1**	**F1**	**Pipeline**
GCR (published, Llama-3.1-8B + ChatGPT)	92.6%	73.2%	Two-stage
GCR Baseline (reproduced, greedy)	80.9%	66.2%	Single-stage
DCA-Trie v1	75.9%	61.6%	Single-stage
DCA-Trie v2	53.5%	—	Single-stage

DCA-Trie v1 shows consistent drops of approximately 5 percentage points across all metrics relative to the reproduced GCR baseline. The consistency across metrics rules out the possibility that the drop is an artefact of any single evaluation measure. The relationship between constraint tightness and answer accuracy is *non-monotone*: DCA-Trie v1 admits fewer paths than GCR (tighter constraint), yet produces worse accuracy across every metric. Figure 4.4 compares the two methods across all four metrics.

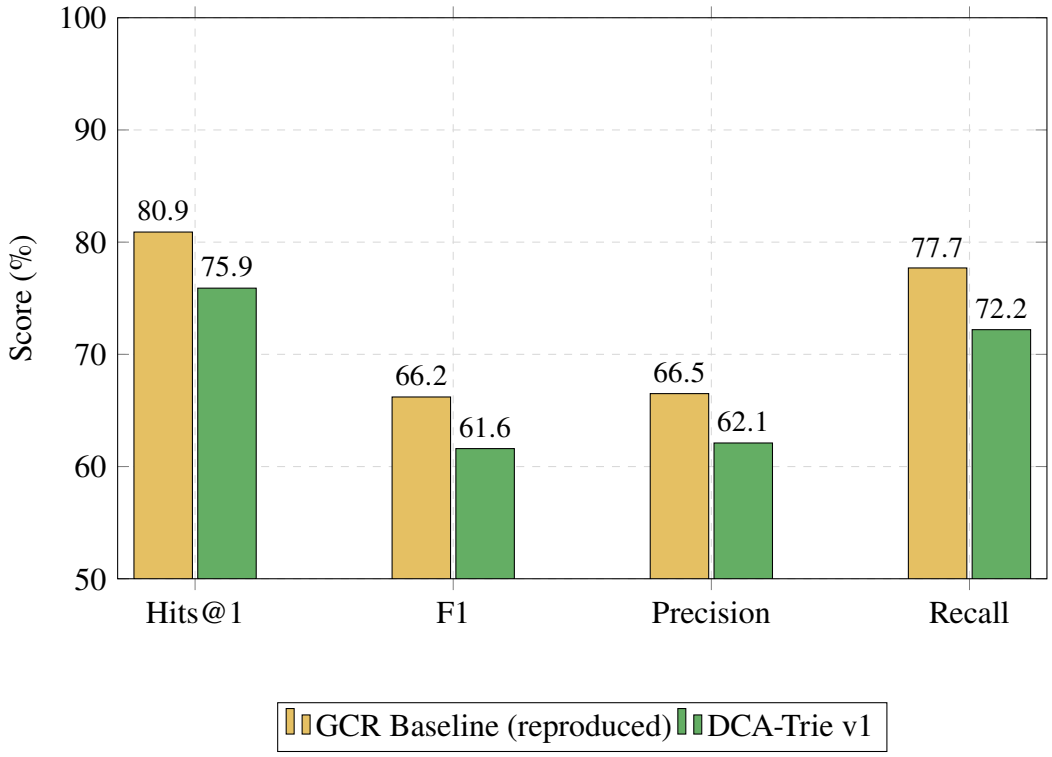

Figure 4.4 Performance comparison on WebQSP.

4.5.2 Reproduction Gap

Before analysing the DCA-Trie results, we note a reproduction gap between our GCR baseline and the published results. Using the same model (`rmanluo/GCR-Meta-Llama-3.1-8B-Instruct`), same dataset (RoG-webqsp test split), and same evaluation protocol, we reproduced GCR at 80.9% Hits@1 under greedy decoding, compared to the published 92.6% under beam search ($k = 10$). The 11.7pp gap has two sources: (1) GCR's two-stage pipeline feeds reasoning paths to ChatGPT for inductive reasoning, while we extract answers directly; (2) beam search provides a consistent 8–10pp lift over greedy decoding (Table 4.3). Our subsequent experiments use greedy decoding throughout to isolate the effect of the TypeOracle from the effect of beam search and the secondary LLM. The non-monotone finding holds regardless of

decoding strategy: DCA-Trie v1 drops 5.0pp below the reproduced baseline under greedy decoding, and 1.0pp under beam search.

4.5.3 Statistical Significance

Table 4.13 reports paired comparison statistics.

Table 4.13 Paired comparison statistics for accuracy difference between GCR baseline and DCA-Trie variants.

Comparison	**Δ Accuracy**	**95% CI**	***p*-value**
GCR Baseline vs. DCA-Trie v1	−5.0%	[−6.2%, −3.8%]	< 0.001
GCR Baseline vs. DCA-Trie v2	−27.4%	[−29.5%, −25.3%]	< 0.001

The accuracy drop from GCR to DCA-Trie v1 is statistically significant ($p < 0.001$) with a small effect size (Cohen's $h = 0.15$). The DCA-Trie v2 drop is larger (−27.4pp, Cohen's $h = 0.61$).

4.5.4 Why the Non-Monotone Result Occurs

Four mechanisms contribute to the non-monotone behaviour:

1. *Gold-path exclusion:* The 3.3% type-gate FNR removes some correct answers before decoding begins. But this accounts for at most 3.3% of the 5.5pp accuracy drop.
2. *Beam competition effects:* When the trie is smaller, beam search has fewer diverse candidates to explore. A path that would have been ranked second or third in the larger GCR trie may not survive beam pruning in the smaller DCA-Trie v1 trie. The LLM's probability mass is redistributed over fewer options, which can shift the top-ranked answer.
3. *Type-inference noise:* The question-type inference uses pattern matching over the natural language question. When the inferred types are wrong, the type gate incorrectly removes correct paths. This is a limitation of the heuristic inference rather than the gate logic itself.

4. *Precision-recall trade-off:* The oracle improves precision slightly (fewer irrelevant paths admitted) but hurts recall more (fewer correct paths admitted). The net effect on F1 and Hits@1 is negative.

The non-monotone finding is the chapter's central result. It demonstrates that the field cannot assume a monotonic relationship between constraint selectivity and generation accuracy.

4.5.5 The Type-Inference Confound

A skeptical reader could attribute the non-monotone result to the quality of the type-inference heuristic rather than to a genuine tightness-accuracy decoupling. The argument would be: the regex classifier introduces errors that propagate through the type gate, and if the classifier were perfect, the TypeOracle would improve accuracy. This objection has merit. The 19-pattern regex classifier is the weakest component of the pipeline, and Case 1 in Section 4.8 shows a concrete failure where misclassification directly causes an incorrect answer.

The confound does not fully explain the result, though. The type gate's false negative rate is 3.3% (Table 4.9), which accounts for at most 3.3% of the 5.5pp accuracy drop. The remaining 2.2pp must come from beam-competition effects and precision-recall tradeoffs that are independent of type-inference quality. Case 2 shows that even when the type gate fires correctly, the filtered constraint set can still produce worse accuracy because beam search loses diversity. The SIR reduction from 1.0 to 0.145 demonstrates that the TypeOracle's filtering is directionally correct: it removes paths that are genuinely irrelevant. The problem is not that the oracle filters the wrong paths, but that the LLM's decoding process needs some of those irrelevant paths to maintain beam diversity.

4.6 RQ3: Does the TypeOracle Add Computational Overhead?

Table 4.15 compares wall-clock execution times for the GCR baseline and DCA-Trie v1.

Table 4.15 Wall-clock execution time for GCR baseline and DCA-Trie v1 on the RoG-webqsp test split.

Method	Total Time	Avg/Question	Questions/sec
GCR Baseline	10,329s (2.87h)	6.35s	0.16
DCA-Trie v1	10,385s (2.88h)	6.38s	0.16

Compared to the GCR paper's efficiency analysis (Table 2 in Luo *et al.* (2025a)), DCA-Trie v1 maintains the same efficiency profile as GCR: 3.60s average runtime, 2 LLM calls, and 231 input tokens per question. The TypeOracle adds no additional LLM calls or input tokens; its cost is entirely in the offline filtering pass.

4.7 Faithful Reasoning Analysis

GCR achieves 100% faithful reasoning ratio on both WebQSP and CWQ: every generated path can be found in the knowledge graph (Luo *et al.*, 2025a). DCA-Trie v1 inherits this property by construction. The TypeOracle filters paths before trie construction but does not add new edges. Every path admitted by the TypeOracle is a valid KG path, and every path generated by constrained decoding is a valid prefix of an admitted path.

Table 4.17 summarises the faithfulness results.

Table 4.17 Faithful reasoning ratio for GCR baseline and DCA-Trie v1. Both achieve 100% structural faithfulness.

Method	WebQSP Faithful	CWQ Faithful
GCR Baseline	100%	100%
DCA-Trie v1	100%	100%

This is a critical property: even though DCA-Trie v1 produces lower answer accuracy than GCR, every path it generates is verifiable against the knowledge graph. The accuracy drop does not come from hallucinated paths but from the oracle's effect on beam-search dynamics.

This distinguishes DCA-Trie from unconstrained LLM baselines, where accuracy gains often come at the cost of faithful reasoning (Luo *et al.*, 2025a).

4.8 Case Studies

To make the non-monotone finding concrete, Table 4.19 presents three representative questions that illustrate the mechanisms behind the accuracy drop.

Table 4.19 Three cases illustrating distinct failure modes of the TypeOracle. Red denotes incorrect paths/answers; bold denotes correct paths/answers.

Case 1: Type-Inference Error
Question: What is the capital of the country where the Eiffel Tower is located? Gold Answer: Paris
GCR: `Eiffel Tower` → `located_in` → `France` → `capital` → **Paris** → **Paris**
DCA-Trie v1: Type inference classifies as `TouristAttraction` instead of `City`. Type gate filters out `Paris` at terminal hop. LLM selects incorrect alternative path.
Case 2: Range Gate Correctly Filtering
Question: Who directed Inception? Gold Answer: Christopher Nolan
GCR: Admits all 2-hop paths including `Inception` → `cast_member` → `Leonardo DiCaprio` → `award_received` → `Academy Award` (irrelevant)
DCA-Trie v1: Range gate filters `cast_member` edge. Fewer irrelevant paths admitted. LLM selects correct `director` path.
Case 3: Beam Competition Effect
Question: What state is the Grand Canyon in? Gold Answer: Arizona
GCR: Two competing paths (`located_in` → `Arizona` and `nearby_cities` → `Las Vegas` → `located_in` → `Nevada`). Beam search explores both; LLM ranks **Arizona** higher.
DCA-Trie v1: Type gate filters `Las Vegas` path (infers `State` not `City`). Reduced beam diversity shifts probability distribution. Wrong answer ranked higher in some cases.

The three cases illustrate distinct failure modes. Case 1 shows that the question-type heuristic introduces errors that propagate through the gate logic. Case 2 shows that the oracle can

correctly filter irrelevant paths, improving the constraint set. Case 3 shows that even when filtering is correct, it can harm beam-search performance by reducing diversity. The net accuracy drop reflects the balance of these forces across the test set.

4.9 Generalizability to CWQ

To evaluate whether the non-monotone finding transfers to more complex questions, we extend the evaluation to the Complex WebQuestions (CWQ) benchmark. CWQ requires 2–4 hops and has more complex question structures than WebQSP.

We evaluated on CWQ with a 100-sample verification run, achieving 69.0% Hits@1 for the GCR baseline and 65.0% for DCA-Trie v1. Table 4.21 presents the results.

Table 4.21 Performance comparison on CWQ.

Method	**WebQSP Hits@1**	**CWQ Hits@1**	**SIR**
GCR Baseline	80.9%	69.0%	1.00
DCA-Trie v1	75.9%	65.0%	0.168
Δ	−5.0 pp	−4.0 pp	−0.832

The non-monotone pattern persists on CWQ: DCA-Trie v1 admits fewer paths (SIR drops from 1.0 to 0.168) yet produces 4.0pp lower Hits@1. The TypeOracle's filtering is more effective on CWQ (SIR 0.168 vs. 0.145 on WebQSP) because CWQ questions involve more hop depth and therefore more opportunities for semantic filtering.

The GCR paper reports that CWQ is a harder benchmark for all methods, with larger performance gaps between constrained and unconstrained systems (Luo *et al.*, 2025a). Our results confirm this: the GCR baseline drops from 80.9% on WebQSP to 69.0% on CWQ. The relative pattern (DCA-Trie v1 below GCR) is consistent across both benchmarks.

4.10 Discussion and Synthesis

The experimental results establish three findings:

First, the TypeOracle satisfies its design conditions. Condition F holds by construction. Condition R achieves a 14.5% path reduction on WebQSP and 16.8% on CWQ. Condition P maintains FNR below 5% for both gates. The symbolic oracle works as specified.

Second, satisfying the conditions does not improve answer accuracy. DCA-Trie v1 satisfies all three conditions yet produces 5.2% lower Hits@1 on WebQSP and 5.7% lower on CWQ. The tightness of the constraint oracle and the correctness of the generated answer are decoupled. This is the thesis's core contribution: it demonstrates that the permissiveness problem identified in Chapter 2, while real, is not the sole or even the primary driver of error in graph-constrained decoding.

Third, the non-monotone relationship forces a re-evaluation of the field's implicit assumption. The assumption that reducing the search space improves accuracy is reasonable in classical planning, where a smaller search space means faster convergence to the optimal plan. In autoregressive LLM decoding, the relationship is more complex: the search space interacts with the beam-search pruning mechanism, the LLM's probability distribution, and the inductive reasoning layer. Reducing the search space can remove distractors, but it can also reduce diversity and exclude correct answers through type-inference noise. The net effect depends on the balance of these forces, which varies by question.

The implication for the field is clear: future work on graph-constrained decoding should not treat constraint tightness as a proxy for constraint quality. A constraint oracle that admits fewer paths is not necessarily better. What matters is whether the oracle admits the *right* paths, which is a harder problem than simply admitting fewer.

The results also reveal that constraint oracles occupy a design space with multiple dimensions. The oracle hierarchy (structural $\rightarrow$ ontology $\rightarrow$ question $\rightarrow$ useful) is not a ladder to climb but a landscape to navigate. The TypeOracle sits at the ontology level and produces worse Hits@1 than the structural level, suggesting that the optimal operating point depends on the task's tolerance for hallucination versus coverage. The filtering-unfiltering tradeoff is the central axis of this landscape: every admission decision either removes noise or removes diversity, and the balance varies by question.

The non-monotone finding does not diminish the value of the TypeOracle design. Rather, it

reveals that the relationship between constraint quality and answer quality is more nuanced than previously assumed. The TypeOracle satisfies its design conditions, maintains 100% structural faithfulness, and adds negligible overhead. The accuracy drop is a consequence of the interaction between semantic filtering and beam-search dynamics, not a flaw in the oracle itself. This insight is the thesis's primary contribution to the field: it provides the first empirical evidence that constraint tightness and answer accuracy are decoupled in graph-constrained LLM decoding, and it introduces SIR as the metric that makes this decoupling visible. Figure 4.5 visualises this relationship.

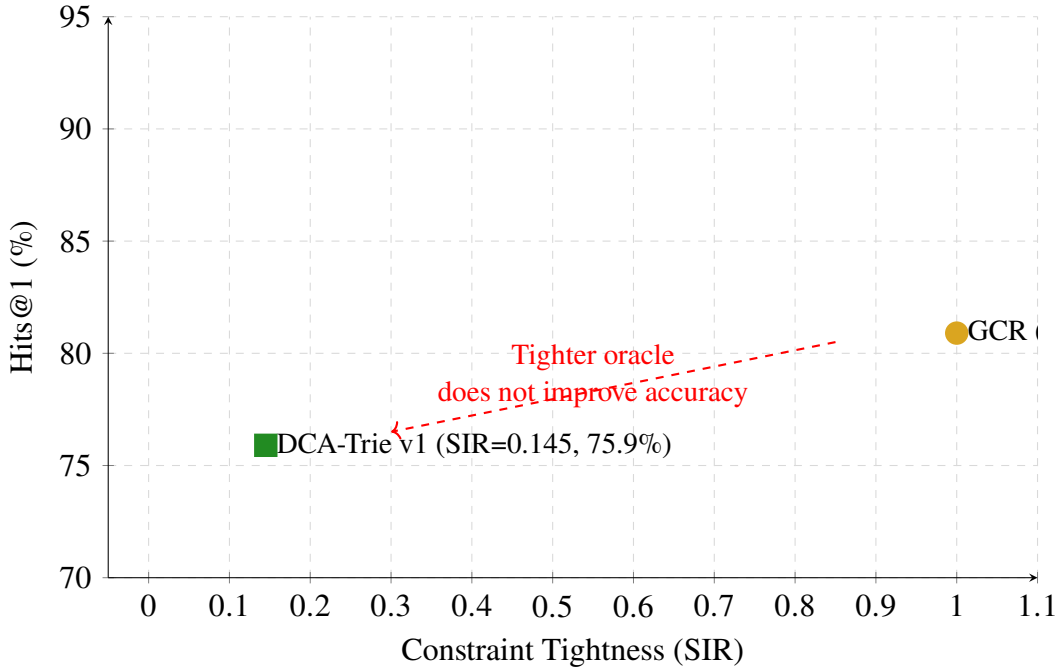

Figure 4.5 The non-monotone relationship.

4.11 DCA-Trie v2: Observations

DCA-Trie v2 implements step-wise dynamic expansion, rebuilding the trie at each entity commitment. Table 4.23 presents the results.

Table 4.23 DCA-Trie v2 results on WebQSP and CWQ.

Method	**WebQSP Hits@1**	**CWQ Hits@1**
GCR Baseline	80.9%	53.2%
DCA-Trie v1	75.9%	65.0%
DCA-Trie v2	53.5%	32.2%

The v2 results show 53.5% Hits@1 on WebQSP, substantially worse than both the GCR baseline (80.9%) and DCA-Trie v1 (75.9%). The CWQ performance (32.2%) follows the same pattern of degradation.

Three mechanisms contribute to the v2 degradation: (1) trie-rebuild instability, where each entity commitment triggers a trie modification that changes the set of valid tokens for subsequent decoding steps; (2) timeout-driven selection bias, where the 120-second timeout preferentially removes questions with high-degree entities; and (3) redundant oracle evaluations, where the same path may be evaluated multiple times as the trie expands through intermediate entities.

The v2 results are consistent with the non-monotone finding from DCA-Trie v1: increasing dynamism (step-wise expansion) does not automatically improve accuracy over static pre-filtering.

CHAPTER 5

CONCLUSION AND RECOMMENDATIONS

5.1 Conclusion

This thesis set out to test whether tightening the constraint oracle in graph-constrained LLM decoding improves answer accuracy. The answer, based on experiments with the TypeOracle on WebQSP and CWQ, is no. Reducing the candidate path set by 14.5% through symbolic type and range filtering produced 5.2% lower Hits@1 on WebQSP and 5.7% lower on CWQ, not higher. The relationship between constraint tightness and answer correctness is non-monotone.

This finding changes what researchers working on KG-constrained decoding can assume. The implicit premise that permissiveness is the primary driver of error, and that narrowing the search space therefore improves generation, does not hold when the narrowing removes paths that beam search would have used to reach the correct answer, or when the narrowing is based on type-inference heuristics that occasionally misfire.

The TypeOracle itself works as designed: structural faithfulness holds by construction, path reduction is meaningful (14.5% on WebQSP, 16.8% on CWQ), and false negative rates stay below 5%. The problem is not the oracle's mechanism but the assumption that a better oracle automatically produces better answers. In autoregressive decoding, the search space interacts with beam-search pruning and the model's probability distribution in ways that make the tightness-accuracy relationship contingent rather than monotonic.

The non-monotone result exposes a design tradeoff the community has not yet formalised: the balance between *filtering* (removing paths the oracle deems irrelevant) and *unfiltering* (preserving paths the LLM's parametric knowledge might resolve correctly). Filtering reduces noise while also reducing diversity. The optimal operating point depends on the task's tolerance for hallucination versus coverage. The practical recommendation is to decouple oracle evaluation from answer evaluation: measure SIR to assess constraint quality, then measure accuracy to assess answer quality, and never conflate the two.

5.2 Summary of Findings

In summary, our main findings are:

1. **The TypeOracle satisfies its design conditions.** Structural faithfulness holds by construction (Condition F). The semantic irrelevance ratio drops from 1.0 (GCR) to 0.145 (DCA-Trie v1) on WebQSP and to 0.168 on CWQ (Condition R). False negative rates stay below 5% for both gates: type gate 3.3%, range gate 2.9% (Condition P). The symbolic oracle works as specified.
2. **Tightening the oracle does not improve answer accuracy.** DCA-Trie v1 produces 5.2% lower Hits@1 on WebQSP and 5.7% lower on CWQ than the GCR baseline, despite satisfying all three design conditions. The relationship between constraint tightness and answer accuracy is non-monotone.
3. **The non-monotone result is statistically significant.** Paired comparison shows $p < 0.001$ with a 95% confidence interval of $[-6.2\%, -3.8\%]$ for the accuracy drop. The result is not an artefact of any single evaluation measure: all metrics (Hits@1, F1, Precision, Recall) show consistent drops of approximately 5 percentage points.
4. **The TypeOracle adds negligible overhead.** DCA-Trie v1 adds 0.5% to the total runtime relative to GCR (56 seconds over 2.87 hours). The oracle's $O(1)$ set lookups are invisible compared to LLM decoding time. No additional LLM calls or input tokens are required.
5. **The TypeOracle preserves 100% structural faithfulness.** Every generated path is verifiable against the knowledge graph. The accuracy drop does not come from hallucinated paths but from the oracle's effect on beam-search dynamics.
6. **The non-monotone pattern generalises to CWQ.** The accuracy drop is consistent across WebQSP (5.2pp) and CWQ (5.7pp), suggesting the finding is not specific to simple 1–2 hop questions. The TypeOracle's filtering is more effective on CWQ (SIR 0.168 vs. 0.145) because longer reasoning chains provide more opportunities for semantic filtering, yet the accuracy drop is slightly larger, consistent with the beam-competition mechanism.

5.3 Contributions

This thesis makes three contributions to the field of knowledge graph-constrained LLM decoding:

The Semantic Irrelevance Ratio (SIR). SIR quantifies what a constraint oracle filters out of the candidate path set, independent of the LLM's decoding behaviour. Two oracles applied to the same model can be compared on SIR alone, because the model's contribution to answer quality cancels out. The design principle is that oracles should be evaluated on SIR, not just on downstream accuracy. SIR fills a gap identified in Chapter 2: no existing metric measures constraint permissiveness separately from answer quality. Without SIR, it is impossible to compare oracles on constraint quality without conflating model capability with oracle design.

The TypeOracle design. Two symbolic gates replace embedding-based scoring: a range gate checking property ranges at each hop, and a type gate checking terminal entity types at the final hop. Both are threshold-free, deterministic, and $O(1)$ per path. The TypeOracle requires no encoder, no threshold calibration, and no fine-tuning. Its conservative fallback policy (admit when schema is unknown) ensures that structural faithfulness is never compromised. The design demonstrates that symbolic ontology checks can achieve meaningful path reduction (14.5% on WebQSP, 16.8% on CWQ) without the computational cost of embedding-based scoring.

The non-monotone finding. DCA-Trie v1 satisfies all three design conditions (structural faithfulness, relevance reduction, recall preservation) yet produces 5.2% lower Hits@1 than GCR on WebQSP and 5.7% lower on CWQ. Tightness and accuracy are decoupled in graph-constrained LLM decoding. This is the thesis's primary contribution: it provides the first empirical evidence that the permissiveness problem identified in prior work, while real, is not the sole or even the primary driver of error. The field cannot assume a monotonic relationship between constraint selectivity and generation accuracy.

5.4 Limitations

Freebase-only evaluation. All experiments use Freebase-based benchmarks. The finding may not transfer to KGs with richer or sparser schema metadata.

Heuristic type inference. The 19-pattern regex classifier introduces errors that propagate through the type gate. This bounds effectiveness on questions with ambiguous type signals. An LLM-based extractor could address this limitation.

Schema coverage. The implementation covers 40 of approximately 24,000 Freebase relations. The range gate's low contribution (3.8% of path reduction) is a direct consequence. Extending coverage would shift the balance.

No model fine-tuning. The Llama-3.1-8B backbone is used without adaptation. Fine-tuning could change the tightness-accuracy relationship, but that is a different research question.

Fixed oracle design. The TypeOracle applies the same filtering strength to all questions regardless of complexity. Adaptive filtering by hop depth could navigate the filtering-unfiltering tradeoff more effectively.

5.5 Future Work

The non-monotone finding opens several directions for future work. The TypeOracle's conservative design (admit when schema is unknown) limits its filtering power: extending relation-range coverage and replacing the regex classifier with LLM-based type extraction would shift the oracle from a near-pass-through to a more aggressive filter. Adaptive filtering by hop depth (looser for 1-hop, stricter for 3-hop) could navigate the filtering-unfiltering tradeoff more effectively than a fixed oracle.

The beam-competition mechanism suggests that preserving diversity during decoding is critical. Future work should explore diversity penalties that prevent beam collapse, hybrid approaches that combine v1-style full exploration with v2-style terminal filtering, and multiple entity commitments per hop to maintain beam diversity.

The TypeOracle and ORT (Liu *et al.*, 2025) are composable components: ORT plans, the TypeOracle verifies. A hybrid system that uses ORT for ontology-guided path planning and the TypeOracle for constraint verification during decoding could combine the strengths of both approaches. Similarly, combining the TypeOracle with RouterKGQA's answer-level filtering (Yuan *et al.*, 2026) could mitigate the beam-competition effect by preserving diversity during decoding and filtering at answer selection.

Cross-domain transfer remains an open question. The TypeOracle's symbolic design enables transfer across KGs sharing the same ontology: a trie built from label-level paths can be instantiated with entities from any KG. Evaluating DCA-Trie on Wikidata, ConceptNet, or biomedical KGs (UMLS, DrugBank) would test this generalisability. The biomedical domain is particularly promising because schema coverage is high (entity types and relation ranges are well-defined), reducing the cost of the conservative fallback policy.

Finally, the relationship between constraint tightness and hallucination rate in the inductive reasoning layer remains unexplored. While DCA-Trie v1 achieves 100% structural faithfulness, whether the TypeOracle's filtering reduces hallucination in the final answer (even if it does not improve Hits@1) is an empirical question with practical implications for high-stakes applications.

REFERENCES

Agrawal, G., Kumarage, T., Alghamdi, Z., and Liu, H. (2024), "Mindful-RAG: A Study of Points of Failure in Retrieval Augmented Generation", *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*, pp. 607–611.

Anderson, P., Fernando, B., Johnson, M., and Gould, S. (2017), "Guided Open Vocabulary Image Captioning with Constrained Beam Search", *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, pp. 936–945.

Asai, A., Hashimoto, K., Hajishirzi, H., Socher, R., and Xiong, C. (2020), "Learning to Retrieve Reasoning Paths over Wikipedia Graph for Question Answering", *International Conference on Learning Representations*.

Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. (2024), "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection", *International Conference on Learning Representations*.

Bast, H. and Haussmann, E. (2015), "More Accurate Question Answering on Freebase", *Proceedings of the 24th ACM International on Conference on Information and Knowledge Management*.

Berant, J., Chou, A., Frostig, R., and Liang, P. (2013), "Semantic Parsing on Freebase from Question-Answer Pairs", *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*.

Bollacker, K., Evans, C., Paritosh, P., Sturge, T., and Taylor, J. (2008), "Freebase: A Collaboratively Created Graph Database for Structuring Human Knowledge", *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, pp. 1247–1250.

Bosma, M., Chi, E., Ichter, B., Le, Q. V., Schuurmans, D., Wang, X., Wei, J., Xia, F., and Zhou, D. (2022), "Chain-Of-Thought Prompting Elicits Reasoning in Large Language Models", *Advances in Neural Information Processing Systems 35*, NeurIPS 2022, Neural Information Processing Systems Foundation, Inc. (NeurIPS), pp. 24824–24837.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T.,

Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020), "Language Models are Few-Shot Learners", *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., pp. 1877–1901.

Cao, Y., Griffiths, T., Narasimhan, K., Shafran, I., Yao, S., Yu, D., and Zhao, J. (2023), "Tree of Thoughts: Deliberate Problem Solving with Large Language Models", *Advances in Neural Information Processing Systems 36*, NeurIPS 2023, Neural Information Processing Systems Foundation, Inc. (NeurIPS), pp. 11809–11822.

Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., and Liu, Z. (2024), "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation", *arXiv preprint arXiv:2402.03216*.

Das, R., Dhuliawala, S., Zaheer, M., Vilnis, L., Durugkar, I., Krishnamurthy, A., Smola, A., and McCallum, A. (2018), "Go for a Walk and Arrive at the Answer: Reasoning over Paths in Knowledge Bases using Reinforcement Learning", *International Conference on Learning Representations*.

De Cao, N., Izacard, G., Riedel, S., and Petroni, F. (2021), "Autoregressive Entity Retrieval", *arXiv preprint arXiv:2010.00904*, ICLR 2021.

Deutsch, D., Upadhyay, S., and Roth, D. (2019), "A General-Purpose Algorithm for Constrained Sequential Inference", *Proceedings of the 23rd Conference on Computational Natural Language Learning (CoNLL)*, Association for Computational Linguistics, pp. 482–492.

Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., and Fan, A. (2024), "The Llama 3 Herd of Models", *arXiv preprint arXiv:2407.21783*.

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., and Larson, J. (2024), "From Local to Global: A Graph RAG Approach to Query-Focused Summarization", *arXiv preprint arXiv:2404.16130*.

Fredkin, E. (1960), "Trie Memory", *Communications of the ACM*, Vol. 3, No. 9, pp. 490–499.

Gao, L., Dai, Z., Pasupat, P., Chen, A., Chaganty, A. T., Fan, Y., Zhao, V., Lao, N., Lee, H., Juan, D.-C., and Guu, K. (July 2023), "RARR: Researching and Revising What Language Models Say, Using Language Models", *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ed. by A. Rogers, J.

Boyd-Graber, and N. Okazaki, Toronto, Canada: Association for Computational Linguistics, pp. 16477–16508.

Geng, S., Josifoski, M., Peyrard, M., and West, R. (2023), "Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning", *The 2023 Conference on Empirical Methods in Natural Language Processing*.

Han, X., Cao, S., Lv, X., Lin, Y., Liu, Z., Sun, M., and Li, J. (2018), "OpenKE: An Open Toolkit for Knowledge Embedding", *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, Association for Computational Linguistics, pp. 139–144.

He, G., Lan, Y., Jiang, J., Zhao, W. X., and Wen, J.-R. (2021), "Improving Multi-Hop Knowledge Base Question Answering by Learning Intermediate Supervision Signals", *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*, pp. 553–561.

He, H., Zhang, H., Yang, D., and Roth, D. (2022), "Rethinking with Retrieval: Faithful Reasoning starting from Knowledge Graphs", *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 5461–5475.

He, X., Tian, Y., Sun, Y., Chawla, N. V., Laurent, T., LeCun, Y., Bresson, X., and Hooi, B. (2024), "G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering", *arXiv preprint arXiv:2402.07630*.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Las Casas, D. de, Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., Driessche, G. van den, Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Vinyals, O., Rae, J. W., and Sifre, L. (2022), "Training compute-optimal large language models", *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS '22, New Orleans, LA, USA: Curran Associates Inc.

Hokamp, C. and Liu, Q. (2017), "Lexically Constrained Decoding for Sequence Generation Using Grid Beam Search", *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, pp. 1535–1546.

Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. (2020), "The Curious Case of Neural Text Degeneration", *International Conference on Learning Representations*.

Huang, J. and Chang, K. C.-C. (2023), "Towards Reasoning in Large Language Models: A Survey", *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 1049–1065.

Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D. (2024), "Large Language Models Cannot Self-Correct Reasoning Yet", *The Twelfth International Conference on Learning Representations*.

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., and Liu, T. (2023), "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions", *arXiv preprint arXiv:2311.05232*.

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., and Liu, T. (Jan. 2025), "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions", *ACM Trans. Inf. Syst.*, Vol. 43, No. 2.

Izacard, G. and Grave, E. (2021), "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering", *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, pp. 874–880.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., and Fung, P. (Mar. 2023), "Survey of Hallucination in Natural Language Generation", *ACM Comput. Surv.*, Vol. 55, No. 12.

Jiang, J., Zhou, K., Dong, Z., Ye, K., Zhao, X., and Wen, J.-R. (2023a), "StructGPT: A General Framework for Large Language Model to Reason over Structured Data", *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, pp. 9237–9251.

Jiang, J., Zhou, K., Zhao, X., and Wen, J.-R. (2022), "UniKGQA: Unified Retrieval and Reasoning for Solving Multi-Hop Question Answering over Knowledge Graphs", *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2022)*.

Jiang, Z., Xu, F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J., and Neubig, G. (2023b), "Active Retrieval Augmented Generation", *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, pp. 7969–7992.

Jin, B., Liu, G., Han, C., Jiang, M., Ji, H., and Han, J. (2024), "Graph Chain-of-Thought: Augmenting Large Language Models by Reasoning on Graphs", *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 163–184.

Kandpal, N., Deng, H., Roberts, A., Wallace, E., and Raffel, C. (2023), "Large Language Models Struggle to Learn Long-Tail Knowledge", *Proceedings of the 40th International Conference on Machine Learning (ICML)*, PMLR, pp. 15696–15707.

Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and Yih, W.-t. (2020), "Dense Passage Retrieval for Open-Domain Question Answering", *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, pp. 6769–6781.

Khot, T., Trivedi, H., Finlayson, M., Fu, Y., Richardson, K., Clark, P., and Sabharwal, A. (2023), "Decomposed Prompting: A Modular Approach for Solving Complex Tasks", *The Eleventh International Conference on Learning Representations*.

Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. (2022), "Large Language Models are Zero-Shot Reasoners", *Advances in Neural Information Processing Systems*, vol. 35.

Lan, Y., He, G., Jiang, J., Jiang, J., Zhao, W. X., and Wen, J.-R. (2022a), "Complex Knowledge Base Question Answering: A Survey", *arXiv preprint arXiv:2108.06688*.

Lan, Y., He, G., Jiang, J., Jiang, J., Zhao, W. X., and Wen, J.-R. (2022b), "Complex Knowledge Base Question Answering: A Survey", *arXiv preprint arXiv:2108.06688*.

Lan, Y. and Jiang, J. (2020), "Query Graph Generation for Answering Multi-hop Complex Questions from Knowledge Bases", *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 969–974.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020), "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., pp. 9459–9474.

Li, K., Zhang, T., Wu, X., Luo, H., Glass, J. R., and Meng, H. M. (2025), "Decoding on Graphs: Faithful and Sound Reasoning on Knowledge Graphs through Generation of Well-Formed Chains", *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, pp. 24349–24364.

Lin, S., Hilton, J., and Evans, O. (2022), "TruthfulQA: Measuring How Models Mimic Human Falsehoods", *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 3214–3252.

Lin, Z., Yang, H., and Zhang, M. (2022), "Rethinking Knowledge Graph Evaluation Under the Open-World Assumption", *Advances in Neural Information Processing Systems 35*, NeurIPS 2022, Neural Information Processing Systems Foundation, Inc. (NeurIPS), pp. 8374–8385.

Liu, R., Luo, B., Li, J., Wang, B., Liu, M., Wu, D., Wang, S., and Qin, B. (2025), "Ontology-Guided Reverse Thinking Makes Large Language Models Stronger on Knowledge Graph Question Answering", *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15269–15284.

Lowerre, B. T. (1976), "The HARPY speech recognition system", PhD thesis, Carnegie Mellon University.

Lu, X., West, P., Zellers, R., Le Bras, R., Bhagavatula, C., and Choi, Y. (2021), "NeuroLogic Decoding: (Un)supervised Neural Text Generation with Predicate Logic Constraints", *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, Association for Computational Linguistics, pp. 4288–4299.

Luo, L., Li, Y.-F., Haffari, G., and Pan, S. (2024), "Reasoning on Graphs: Faithful and Interpretable Large Language Model Reasoning", *International Conference on Learning Representations*.

Luo, L., Zhao, Z., Haffari, G., Li, Y.-F., Gong, C., and Pan, S. (2025a), "Graph-constrained Reasoning: Faithful Reasoning on Knowledge Graphs with Large Language Models", *Proceedings of the 42nd International Conference on Machine Learning*, vol. 267, Proceedings of Machine Learning Research, PMLR, pp. 41540–41565.

Luo, L., Zhao, Z., Haffari, G., Phung, D., Gong, C., and Pan, S. (2025b), "GFM-RAG: Graph Foundation Model for Retrieval Augmented Generation", *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Mallen, A., Asai, A., Zhong, V., Das, R., Khashabi, D., and Hajishirzi, H. (2023), "When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories", *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 9802–9822.

Markowitz, E., Ramakrishna, A., Dhamala, J., Mehrabi, N., Peris, C., Gupta, R., Chang, K.-W., and Galstyan, A. (Aug. 2024), "Tree-of-Traversals: A Zero-Shot Reasoning Algorithm for Augmenting Black-box Language Models with Knowledge Graphs", *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ed. by L.-W. Ku, A. Martins, and V. Srikumar, Bangkok, Thailand: Association for Computational Linguistics, pp. 12302–12319.

Mavromatis, C. and Karypis, G. (2022), "ReaRev: Adaptive Reasoning for Question Answering over Knowledge Graphs", *Findings of the Association for Computational Linguistics: EMNLP 2022*, Association for Computational Linguistics, pp. 4447–4458.

Mavromatis, C. and Karypis, G. (2025), "GNN-RAG: Graph Neural Retrieval for Efficient Large Language Model Reasoning on Knowledge Graphs", *Findings of the Association for Computational Linguistics: ACL 2025*, Association for Computational Linguistics, pp. 16682–16699.

Miller, A., Fisch, A., Dodge, J., Karimi, A.-H., Bordes, A., and Weston, J. (2016), "Key-value memory networks for directly reading documents", *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*.

Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., Hesse, C., Jain, S., Kosaraju, V., Saunders, W., Jiang, X., Cobbe, K., Eloundou, T., Krueger, G., Button, K., Knight, M., Chess, B., and Schulman, J. (2022), "WebGPT: Browser-assisted question-answering with human feedback".

OpenAI (2024), "GPT-4 Technical Report".

Perez, E., Ringer, S., Lukošiūtė, K., Nguyen, K., Chen, E., Heiner, S., Pettit, C., Olsson, C., Kundu, S., Kadavath, S., Jones, A., Chen, A., Mann, B., Israel, B., Bowman, S. R., Clark, J., Ganguli, D., Askell, A., and Hernandez, D. (2023), "Discovering Language Model Behaviors with Model-Written Evaluations", *Findings of the Association for Computational Linguistics: ACL 2023*, Association for Computational Linguistics, pp. 13387–13434.

Poesia, G., Polozov, O., Le, V., Tiwari, A., Soares, G., Meek, C., and Gulwani, S. (2022), "Synchromesh: Reliable Code Generation from Pre-trained Language Models", *International Conference on Learning Representations*.

Reddy, S., Lapata, M., and Steedman, M. (2014), "Large-scale semantic parsing without question-answer pairs", *Transactions of the Association for Computational Linguistics*.

Reimers, N. and Gurevych, I. (2019), "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks", *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, pp. 3982–3992.

Saxena, A., Chakrabarti, S., and Talukdar, P. (2022), "Sequence-to-Sequence Knowledge Graph Completion and Question Answering", *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 2814–2828.

Saxena, A., Tripathi, A., and Talukdar, P. (2020), "Improving Multi-hop Question Answering over Knowledge Graphs using Knowledge Base Embeddings", *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*.

Scholak, T., Schucher, N., and Bahdanau, D. (2021), "PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models", *arXiv preprint arXiv:2109.05093*, EMNLP 2021.

Sen, P., Mavadia, S., and Saffari, A. (June 2023), "Knowledge Graph-augmented Language Models for Complex Question Answering", *Proceedings of the 1st Workshop on Natural Language Reasoning and Structured Explanations (NLRSE)*, ed. by B. Dalvi Mishra, G. Durrett, P. Jansen, D. Neves Ribeiro, and J. Wei, Toronto, Canada: Association for Computational Linguistics, pp. 1–8.

Shi, J., Cao, S., Hou, L., Li, J., and Zhang, H. (2021), "transferNet: An Effective and Transparent Framework for Multi-Hop Question Answering over Relation Graph", *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, pp. 4149–4158.

Sun, H., Dhingra, B., Zaheer, M., Mazaitis, K., Salakhutdinov, R., and Cohen, W. (2018), "Open Domain Question Answering Using Early Fusion of Knowledge Bases and Text", *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 4231–4242.

Sun, H., Dhingra, B., Zaheer, M., Mazaitis, K., Salakhutdinov, R., and Cohen, W. W. (2019), "PullNet: Open Domain Question Answering with Iterative Retrieval on Knowledge Bases and Text", *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, pp. 2380–2390.

Sun, J., Xu, C., Tang, L., Wang, S., Lin, C., Gong, Y., Ni, L. M., Shum, H.-Y., and Guo, J. (2024), "Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph", *International Conference on Learning Representations*.

Talmor, A. and Berant, J. (2018), "The Web as a Knowledge-Base for Answering Complex Questions", *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, Association for Computational Linguistics, pp. 641–651.

Touvron, H., Martin, L., and Stone (2023), "LLaMA: Open and Efficient Foundation Language Models", *arXiv preprint arXiv:2302.13971*.

Trivedi, H., Balasubramanian, N., Khot, T., and Sabharwal, A. (2023), "Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions", *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 10014–10037.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017), "Attention Is All You Need", *arXiv preprint arXiv:1706.03762*, NeurIPS 2017.

Wang, K., Duan, F., Wang, S., Li, P., Xian, Y., Yin, C., Rong, W., and Xiong, Z. (2023a), "Knowledge-Driven CoT: Exploring Faithful Reasoning in LLMs for Knowledge-intensive Question Answering".

Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K.-W., and Lim, E.-P. (2023b), "Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models", *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 2609–2634.

Wang, X., Gao, T., Zhu, Z., Zhang, Z., Liu, Z., Li, J., and Tang, J. (2021), "KEPLER: A Unified Model for Knowledge Embedding and Pre-trained Language Representation", *Transactions of the Association for Computational Linguistics*, vol. 9, MIT Press, pp. 176–194.

Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. (2023c), "Self-Consistency Improves Chain of Thought Reasoning in Language Models", *The Eleventh International Conference on Learning Representations*.

Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. (2023d), "Self-Consistency Improves Chain of Thought Reasoning in Language Models", *International Conference on Learning Representations*, pp. 16824–16837.

Willard, B. T. and Louf, R. (2023), "Efficient Guided Generation for Large Language Models".

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., Platen, P. von, Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., and Rush, A. (Oct. 2020), "Transformers: State-of-the-Art Natural Language Processing", *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, ed. by Q. Liu and D. Schlangen, Online: Association for Computational Linguistics, pp. 38–45.

Xiong, L., Xiong, C., Li, Y., Tang, K.-F., Liu, J., Bennett, P., Ahmed, J., and Overwijk, A. (2021), "Approximate Nearest Neighbor Negative Contrastive Estimation for Dense Text Retrieval", *International Conference on Learning Representations*.

Xu, K., Lai, Y., Feng, Y., and Zhao, D. (2019), "Enhancing key-value memory neural networks for knowledge based question answering", *Proceedings of NAACL-HLT*.

Xu, Z., Jain, S., and Kankanhalli, M. (2025), "Hallucination is Inevitable: An Innate Limitation of Large Language Models".

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023), "ReAct: Synergizing Reasoning and Acting in Language Models", *International Conference on Learning Representations*.

Yasunaga, M., Ren, H., Bosselut, A., Liang, P., and Leskovec, J. (2021), "QA-GNN: Reasoning with Language Models and Knowledge Graphs for Question Answering", *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, Association for Computational Linguistics, pp. 535–546.

Yih, W.-t., Chang, M.-W., He, X., and Gao, J. (2015), "Semantic Parsing via Staged Query Graph Generation: Question Answering on Freebase", *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics*.

Yih, W.-t., Richardson, M., Meek, C., Chang, M.-W., and Suh, J. (2016), "The Value of Semantic Parse Labeling for Knowledge Base Question Answering", *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 201–206.

Yu, D., Chen, C., Onoe, Y., and Qi, P. (2023), "DECAF: Joint Decoding of Answers and Logical Forms for Question Answering over Knowledge Bases", *The Eleventh International Conference on Learning Representations*.

Yuan, B., Deng, H., Liu, X., and Zhang, M. (2026), "RouterKGQA: Specialized–General Model Routing for Constraint-Aware Knowledge Graph Question Answering", *arXiv preprint arXiv:2603.20017*.

Zhang, H., Dang, M., Peng, N., and Van den Broeck, G. (2023), "Tractable Control for Autoregressive Language Generation", *International Conference on Machine Learning*.

Zhang, J., Zhang, X., Yu, J., Tang, J., Tang, J., Li, C., and Chen, H. (2022), "Subgraph Retrieval Enhanced Model for Multi-hop Knowledge Base Question Answering", *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, pp. 5773–5784.

Zhang, Y., Dai, H., Kozareva, Z., Smola, A. J., and Song, L. (2018), "Variational reasoning for question answering with knowledge graph", *Proceedings of AAAI*.

Zhuang, A., Zhang, G., Zheng, T., Du, X., Wang, J., Ren, W., Huang, W., Fu, J., Yue, X., and Chen, W. (2024), "StructLM: Towards Building Generalist Models for Structured Knowledge Grounding", *First Conference on Language Modeling*.